

prediksi-life-expectancy

June 26, 2026

1. PERUMUSAN TUJUAN BISNIS & PENGUMPULAN DATA

1.1 a. Perumusan Tujuan Bisnis

Project ini bertujuan untuk mengembangkan model prediktif yang dapat memperkirakan angka harapan hidup (*Life Expectancy*) suatu negara berdasarkan berbagai faktor seperti kesehatan, ekonomi, dan sosial.

Tujuan utama:

- Memahami Faktor Penentu:** Mengidentifikasi faktor-faktor kunci (misalnya, tingkat kematian dewasa, pengeluaran kesehatan, tingkat pendidikan, dll.) yang paling berpengaruh terhadap harapan hidup.
- Membangun Model Prediktif:** Mengembangkan model regresi yang akurat untuk memprediksi harapan hidup.
- Meningkatkan Pengambilan Keputusan:** Memberikan wawasan yang dapat digunakan oleh pembuat kebijakan dan organisasi kesehatan masyarakat untuk merancang intervensi yang efektif dalam meningkatkan harapan hidup di berbagai negara.

1.2 b. Persiapan Library dan Data

```
[ ]: import os
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
```

```
[ ]: #Memanggil data
file_path = '/content/Life Expectancy Data (1).csv'
df = pd.read_csv(file_path)
# Drop the 'Country', 'Year', and 'Status' columns as requested
#Dihapus karena tidak relevan untuk memprediksi angka harapan hidup
df = df.drop(columns=['Country', 'Year'])
df
```

```
[ ]:      Status  Life expectancy  Adult Mortality  infant deaths  Alcohol  \
0   Developing             65.0             263.0             62      0.01
1   Developing             59.9             271.0             64      0.01
2   Developing             59.9             268.0             66      0.01
```

3	Developing	59.5	272.0	69	0.01
4	Developing	59.2	275.0	71	0.01
...	...	...	...	...	...
2933	Developing	44.3	723.0	27	4.36
2934	Developing	44.5	715.0	26	4.06
2935	Developing	44.8	73.0	25	4.43
2936	Developing	45.3	686.0	25	1.72
2937	Developing	46.0	665.0	24	1.68

	percentage expenditure	Hepatitis B	Measles	BMI	\
0	71.279624	65.0	1154	19.1	
1	73.523582	62.0	492	18.6	
2	73.219243	64.0	430	18.1	
3	78.184215	67.0	2787	17.6	
4	7.097109	68.0	3013	17.2	
...	...	...	...	...	...
2933	0.000000	68.0	31	27.1	
2934	0.000000	7.0	998	26.7	
2935	0.000000	73.0	304	26.3	
2936	0.000000	76.0	529	25.9	
2937	0.000000	79.0	1483	25.5	

	under-five deaths	Polio	Total expenditure	Diphtheria	HIV/AIDS	\
0	83	6.0		65.0	0.1	
1	86	58.0		62.0	0.1	
2	89	62.0		64.0	0.1	
3	93	67.0		67.0	0.1	
4	97	68.0		68.0	0.1	
...	...	...	...	...	...	...
2933	42	67.0		65.0	33.6	
2934	41	7.0		68.0	36.7	
2935	40	73.0		71.0	39.8	
2936	39	76.0		75.0	42.1	
2937	39	78.0		78.0	43.5	

	GDP	Population	thinness 1-19 years	thinness 5-9 years	\
0	584.259210	33736494.0	17.2	17.3	
1	612.696514	327582.0	17.5	17.5	
2	631.744976	31731688.0	17.7	17.7	
3	669.959000	3696958.0	17.9	18.0	
4	63.537231	2978599.0	18.2	18.2	
...	...	...	...	...	...
2933	454.366654	12777511.0	9.4	9.4	
2934	453.351155	12633897.0	9.8	9.9	
2935	57.348340	125525.0	1.2	1.3	
2936	548.587312	12366165.0	1.6	1.7	
2937	547.358878	12222251.0	11.0	11.2	

	Income composition of resources	Schooling
0	0.479	10.1
1	0.476	10.0
2	0.470	9.9
3	0.463	9.8
4	0.454	9.5
...	...	...
2933	0.407	9.2
2934	0.418	9.5
2935	0.427	10.0
2936	0.427	9.8
2937	0.434	9.8

[2938 rows x 20 columns]

1.2.1 Interpretasi Output:

Menampilkan 5 baris pertama dan 5 baris terakhir dari DataFrame `df` yang dimuat dari file `Life Expectancy Data().csv`. Memberikan gambaran awal tentang struktur data, nama-nama kolom (misalnya `Country`, `Year`, `Status`, `Life expectancy`), serta beberapa nilai sampel dari setiap kolom. Terlihat ada 2938 baris dan 22 kolom, mengindikasikan ukuran dataset yang cukup besar dengan berbagai fitur.

2. PENELAAHAN DATA

2.1 a. Mengidentifikasi Tipe Data Setiap Fitur

```
[ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 2938 entries, 0 to 2937
```

```
Data columns (total 20 columns):
```

#	Column	Non-Null Count	Dtype
0	Status	2938 non-null	object
1	Life expectancy	2928 non-null	float64
2	Adult Mortality	2928 non-null	float64
3	infant deaths	2938 non-null	int64
4	Alcohol	2744 non-null	float64
5	percentage expenditure	2938 non-null	float64
6	Hepatitis B	2385 non-null	float64
7	Measles	2938 non-null	int64
8	BMI	2904 non-null	float64
9	under-five deaths	2938 non-null	int64
10	Polio	2919 non-null	float64
11	Total expenditure	2712 non-null	float64

```

12 Diphtheria                2919 non-null   float64
13 HIV/AIDS                  2938 non-null   float64
14 GDP                        2490 non-null   float64
15 Population                 2286 non-null   float64
16 thinness 1-19 years        2904 non-null   float64
17 thinness 5-9 years         2904 non-null   float64
18 Income composition of resources 2771 non-null   float64
19 Schooling                  2775 non-null   float64
dtypes: float64(16), int64(3), object(1)
memory usage: 459.2+ KB

```

2.1.1 Interpretasi Output:

- Terdapat 2938 entri (baris) dalam dataset.
- Kolom 'Country' dan 'Status' bertipe `object` (kategorikal/teks).
- Kolom 'Year' dan 'infant deaths', 'Measles', 'under-five deaths' bertipe `int64`.
- Sebagian besar kolom lainnya bertipe `float64` (numerik).
- Banyak kolom numerik seperti Life expectancy, Adult Mortality, Alcohol, Hepatitis B, BMI, Polio, Total expenditure, Diphtheria, GDP, Population, thinness 1-19 years, thinness 5-9 years, Income composition of resources, dan Schooling memiliki jumlah Non-Null Count yang lebih rendah dari total 2938, menunjukkan adanya **missing values** yang perlu ditangani.

2.2 b. Statistik Deskriptif

```
[ ]: #statistika deskriptif untuk variabel numerik
df.describe()
```

```
[ ]:
      Life expectancy  Adult Mortality  infant deaths  Alcohol  \
count      2928.000000      2928.000000      2938.000000  2744.000000
mean         69.224932        164.796448         30.303948    4.602861
std          9.523867        124.292079        117.926501    4.052413
min         36.300000         1.000000         0.000000    0.010000
25%         63.100000        74.000000         0.000000    0.877500
50%         72.100000       144.000000         3.000000    3.755000
75%         75.700000       228.000000        22.000000    7.702500
max         89.000000       723.000000       1800.000000   17.870000

      percentage expenditure  Hepatitis B      Measles      BMI  \
count      2938.000000      2385.000000      2938.000000  2904.000000
mean         738.251295       80.940461      2419.592240   38.321247
std       1987.914858       25.070016     11467.272489   20.044034
min           0.000000         1.000000         0.000000    1.000000
25%          4.685343       77.000000         0.000000   19.300000
50%          64.912906       92.000000        17.000000   43.500000
75%         441.534144       97.000000       360.250000   56.200000
max       19479.911610       99.000000     212183.000000   87.300000

```

	under-five deaths	Polio	Total expenditure	Diphtheria	\
count	2938.000000	2919.000000	2712.000000	2919.000000	
mean	42.035739	82.550188	5.93819	82.324084	
std	160.445548	23.428046	2.49832	23.716912	
min	0.000000	3.000000	0.37000	2.000000	
25%	0.000000	78.000000	4.26000	78.000000	
50%	4.000000	93.000000	5.75500	93.000000	
75%	28.000000	97.000000	7.49250	97.000000	
max	2500.000000	99.000000	17.60000	99.000000	

	HIV/AIDS	GDP	Population	thinness 1-19 years	\
count	2938.000000	2490.000000	2.286000e+03	2904.000000	
mean	1.742103	7483.158469	1.275338e+07	4.839704	
std	5.077785	14270.169342	6.101210e+07	4.420195	
min	0.100000	1.681350	3.400000e+01	0.100000	
25%	0.100000	463.935626	1.957932e+05	1.600000	
50%	0.100000	1766.947595	1.386542e+06	3.300000	
75%	0.800000	5910.806335	7.420359e+06	7.200000	
max	50.600000	119172.741800	1.293859e+09	27.700000	

	thinness 5-9 years	Income composition of resources	Schooling
count	2904.000000	2771.000000	2775.000000
mean	4.870317	0.627551	11.992793
std	4.508882	0.210904	3.358920
min	0.100000	0.000000	0.000000
25%	1.500000	0.493000	10.100000
50%	3.300000	0.677000	12.300000
75%	7.200000	0.779000	14.300000
max	28.600000	0.948000	20.700000

2.2.1 Interpretasi Output:

Menyajikan statistik deskriptif untuk semua kolom numerik dalam DataFrame, seperti: - **count**: Jumlah nilai non-null. Mengkonfirmasi kembali adanya *missing values* pada banyak kolom. - **mean**: Rata-rata nilai. Misalnya, rata-rata harapan hidup adalah sekitar 69.22 tahun. - **std**: Standar deviasi, mengukur sebaran data. Beberapa fitur seperti **percentage expenditure**, **Measles**, dan **Population** memiliki standar deviasi yang sangat besar, menunjukkan variasi data yang tinggi atau adanya *outlier*. - **min** dan **max**: Nilai minimum dan maksimum. Menarik untuk dicatat bahwa **Life expectancy** berkisar antara 36.3 hingga 89.0 tahun. Beberapa fitur memiliki nilai minimum 0 (misalnya **infant deaths**, **Measles**), yang wajar. - **25%**, **50% (median)**, **75%**: Kuartil, menunjukkan distribusi data. Perbedaan signifikan antara median dan rata-rata, atau antara kuartil dan min/max, dapat mengindikasikan *skewness* (kemiringan distribusi data) atau keberadaan *outlier*.

Secara umum, statistik ini memberikan gambaran awal tentang pusat, sebaran, dan rentang nilai dari setiap fitur numerik.

```
[ ]: #statistik deksriptif untuk variabel kategorik
df.describe(include="object")
```

```
[ ]:      Status
count    2938
unique      2
top    Developing
freq      2426
```

2.2.2 Interpretasi Output:

Memberikan statistik deskriptif untuk kolom non-numerik (kategorikal): - **Country**: Ada 193 negara unik dalam dataset, dengan 'Afghanistan' sebagai negara yang paling sering muncul (frekuensi 16, kemungkinan karena data per tahun dari 2000-2015). - **Status**: Ada 2 status unik: 'Developing' dan 'Developed'. Mayoritas entri (2426 dari 2938) adalah 'Developing', menunjukkan ketidakseimbangan kelas yang signifikan antara negara berkembang dan negara maju dalam dataset ini.

2.3 c. Visualisasi Data

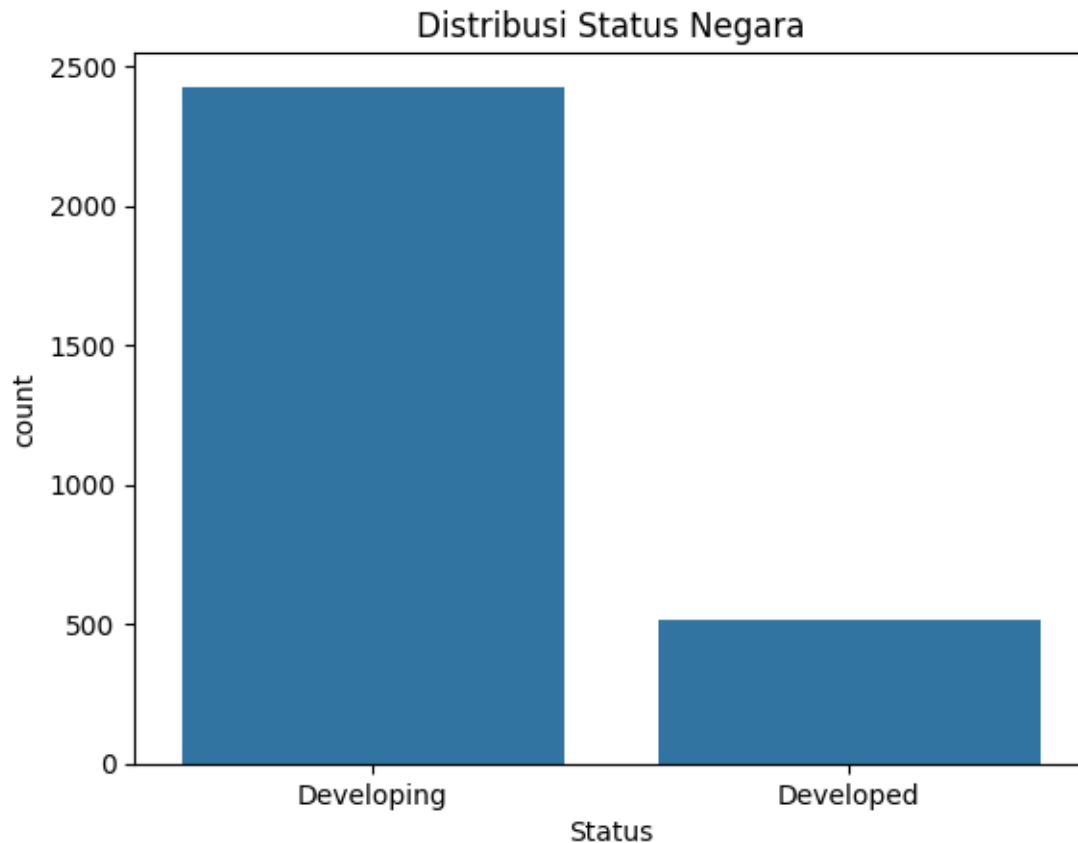
2.3.1 Barchart

```
[ ]: print(df['Status'].value_counts())

sns.countplot(
    x='Status',
    data=df
)

plt.title('Distribusi Status Negara')
plt.show()
```

```
Status
Developing    2426
Developed      512
Name: count, dtype: int64
```



2.3.2 Interpretasi Output:

- Terlihat jelas bahwa ada 2426 entri untuk 'Developing' dan 512 entri untuk 'Developed'.
- Visualisasi ini secara grafis menunjukkan dominasi kategori 'Developing' dibandingkan 'Developed'.

2.3.3 Histogram

```
[ ]: import matplotlib.pyplot as plt
import seaborn as sns
import math

# Identifikasi kolom numerik
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns

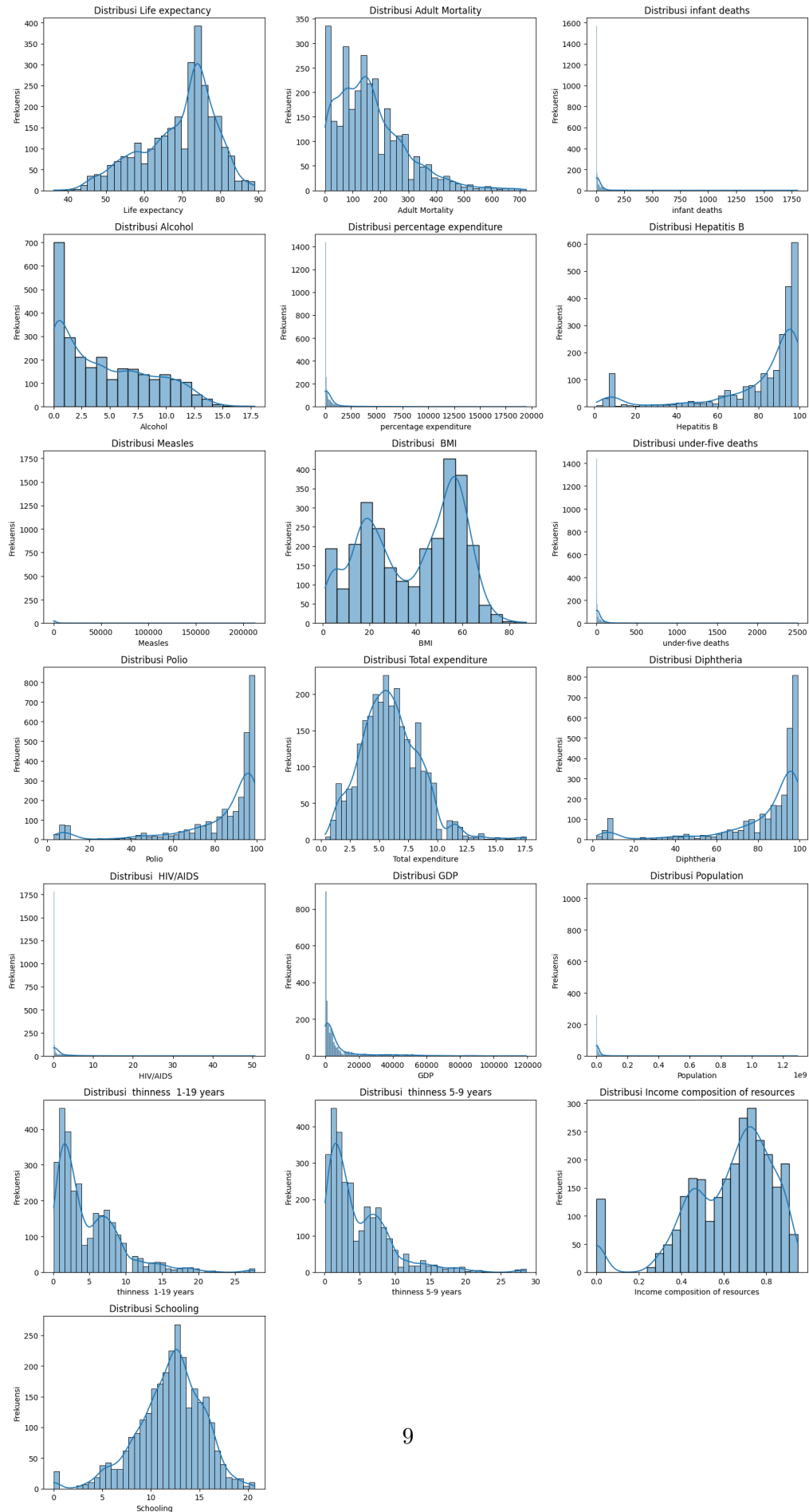
# Filter kolom yang akan diplot
plot_cols = [col for col in numerical_cols]

# Hitung jumlah baris yang dibutuhkan secara dinamis
n_cols = 3
```

```
n_rows = math.ceil(len(plot_cols) / n_cols)

# Buat histogram untuk setiap kolom
plt.figure(figsize=(15, 4 * n_rows))
for i, col in enumerate(plot_cols):
    plt.subplot(n_rows, n_cols, i + 1)
    sns.histplot(df[col].dropna(), kde=True)
    plt.title(f'Distribusi {col}')
    plt.xlabel(col)
    plt.ylabel('Frekuensi')

plt.tight_layout()
plt.show()
```

Interpretasi Output: Kumpulan histogram ini menunjukkan distribusi dari setiap fitur numerik dalam dataset: - **Distribusi normal/mendekati normal:** Beberapa fitur seperti Life expectancy, Adult Mortality, BMI, Schooling, Income composition of resources menunjukkan distribusi yang cenderung simetris atau unimodal. - **Distribusi miring ke kanan (right-skewed):** Banyak fitur menunjukkan distribusi yang sangat miring ke kanan, di mana sebagian besar data terkonsentrasi pada nilai-nilai rendah dan ada “ekor” panjang ke arah nilai tinggi. Contohnya adalah infant deaths, percentage expenditure, Measles, under-five deaths, HIV/AIDS, GDP, dan Population. Ini mengindikasikan keberadaan nilai-nilai ekstrem atau *outlier* yang tinggi.

2.3.4 Box Plot

```
[ ]: import matplotlib.pyplot as plt
import seaborn as sns
import math

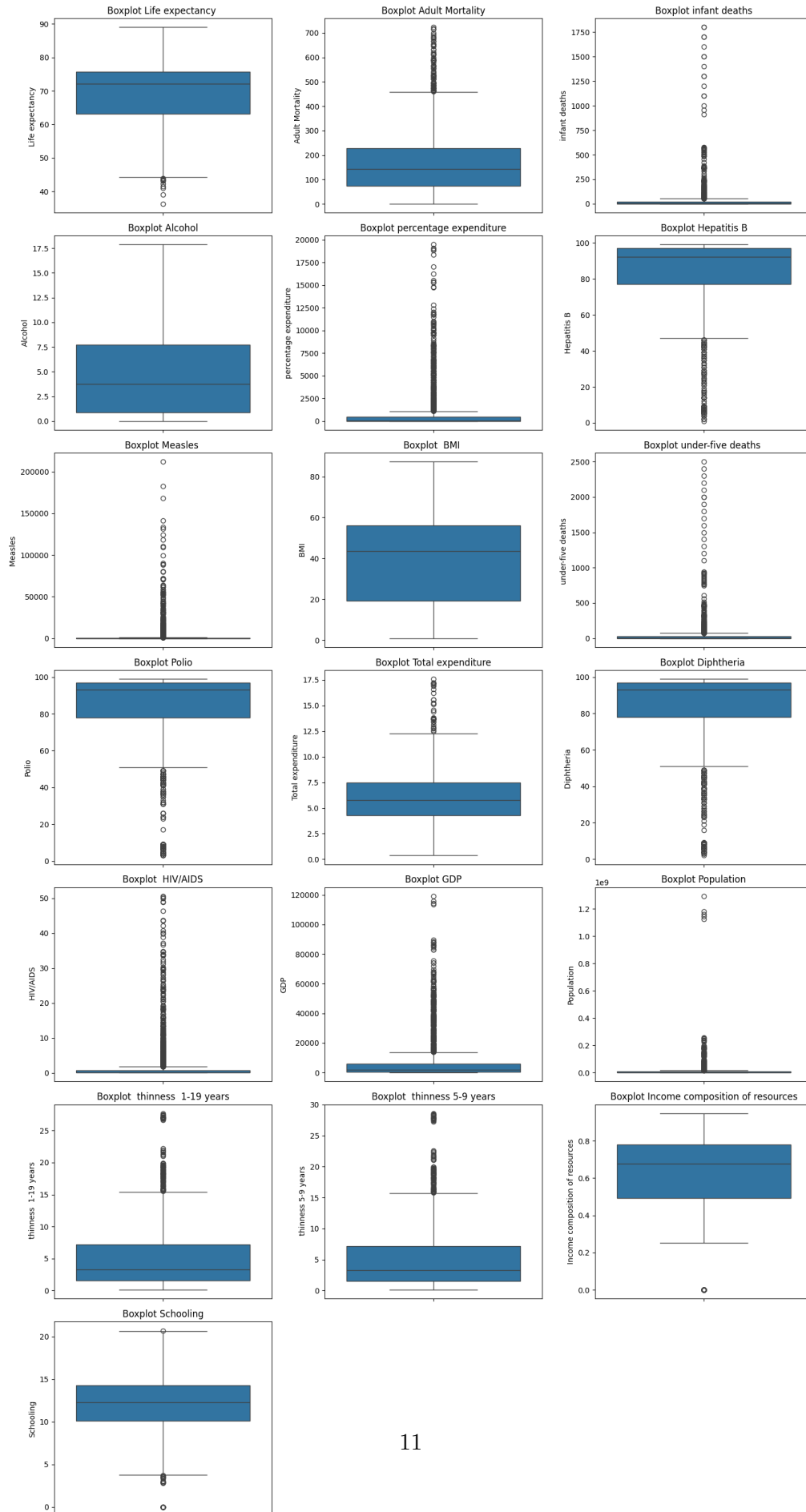
# Identifikasi kolom numerik
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns

# Filter kolom yang akan diplot
plot_cols = [col for col in numerical_cols]

# Hitung jumlah baris yang dibutuhkan secara dinamis agar tidak error (n_cols = 3)
n_cols = 3
n_rows = math.ceil(len(plot_cols) / n_cols)

# Buat boxplot untuk setiap kolom yang tersisa
plt.figure(figsize=(15, 4 * n_rows))
for i, col in enumerate(plot_cols):
    plt.subplot(n_rows, n_cols, i + 1)
    sns.boxplot(y=df[col].dropna()) # Hilangkan nilai NaN untuk plotting
    plt.title(f'Boxplot {col}')
    plt.ylabel(col)

plt.tight_layout()
plt.show()
```



2.3.5 Interpretasi Output:

- **Deteksi Outlier:** Hampir semua *boxplot* mengindikasikan keberadaan *outlier*. Fitur-fitur seperti Adult Mortality, infant deaths, percentage expenditure, Measles, under-five deaths, Polio, Total expenditure, Diphtheria, HIV/AIDS, GDP, Population, thinness 1-19 years, dan thinness 5-9 years memiliki banyak *outlier* yang ekstrim.

2.3.6 Melihat hubungan antar variabel numerik (scatter plot)

```
[ ]: import matplotlib.pyplot as plt
import seaborn as sns
import math

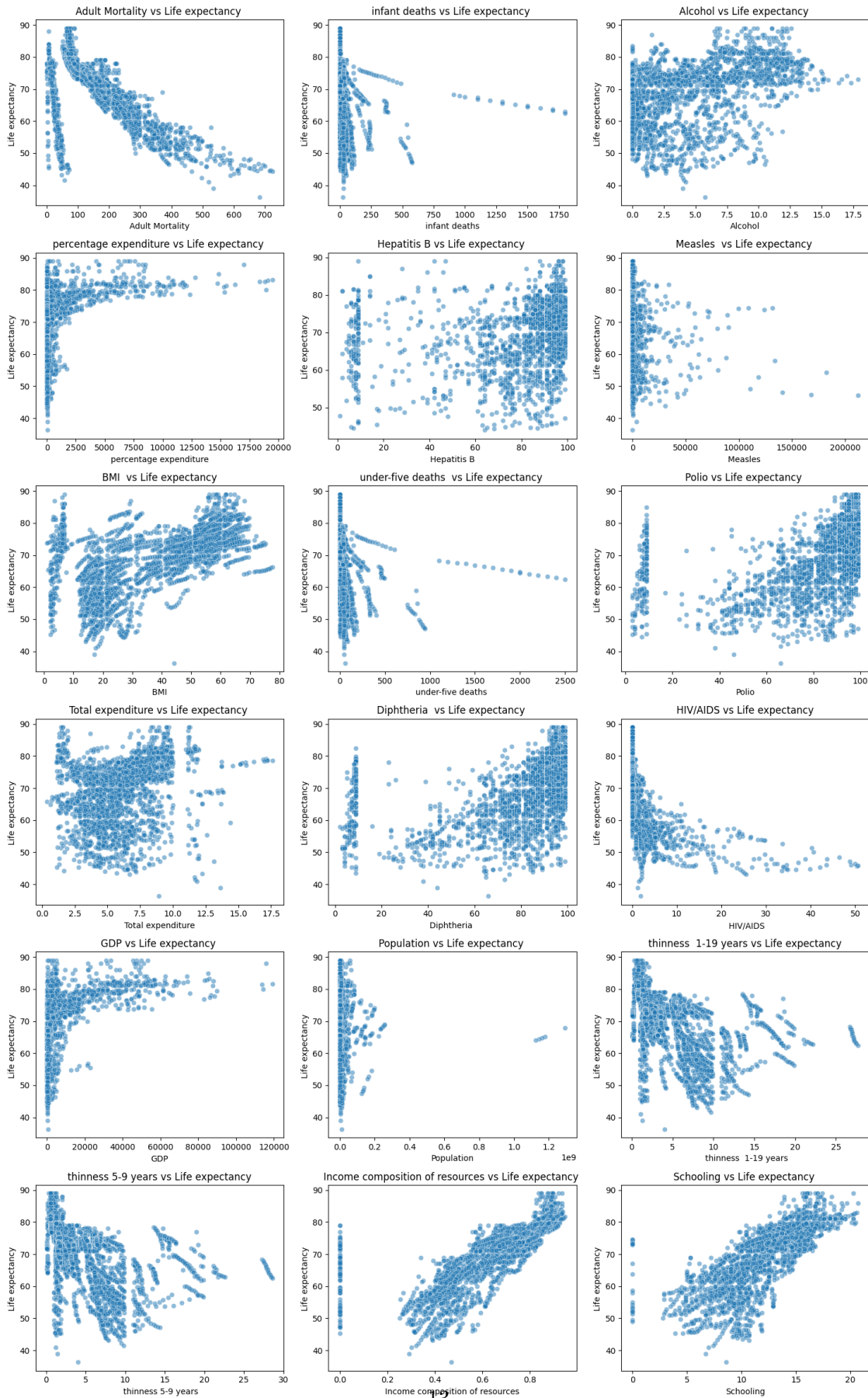
# Mengidentifikasi kolom numerik
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns

# Memperbaiki nama kolom target dan mengeluarkannya dari kolom yang akan diplot
↳ sebagai x-axis
excluded_cols = ['Life expectancy ']
plot_cols = [col for col in numerical_cols if col not in excluded_cols]

# Menghitung jumlah baris grid secara dinamis (3 plot per baris)
n_cols = 3
n_rows = math.ceil(len(plot_cols) / n_cols)

plt.figure(figsize=(15, 4 * n_rows))
for i, col in enumerate(plot_cols):
    plt.subplot(n_rows, n_cols, i + 1)
    sns.scatterplot(x=df[col], y=df['Life expectancy '], alpha=0.5) #
    ↳ Menggunakan nama kolom yang benar
    plt.title(f'{col} vs Life expectancy ') # Menggunakan nama kolom yang benar
    ↳ di judul
    plt.xlabel(col)
    plt.ylabel('Life expectancy ') # Menggunakan nama kolom yang benar di label
    ↳ y

plt.tight_layout()
plt.show()
```



2.3.7 Heatmap matriks korelasi

```
[ ]: #Menampilkan jenis data sebelum transformasi
df.info()
#Membuat objek LabelEncoder
le=LabelEncoder()
#Menentukan kolom kategori yang akan ditransformasi
categorical_cols = ['Status']
#Melakukan transformasi data kategori menjadi numerik
for col in categorical_cols:
    df[col] = le.fit_transform(df[col])
#Menampilkan jenis data setelah transformasi
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 2938 entries, 0 to 2937
```

```
Data columns (total 20 columns):
```

#	Column	Non-Null Count	Dtype
0	Status	2938 non-null	object
1	Life expectancy	2928 non-null	float64
2	Adult Mortality	2928 non-null	float64
3	infant deaths	2938 non-null	int64
4	Alcohol	2744 non-null	float64
5	percentage expenditure	2938 non-null	float64
6	Hepatitis B	2385 non-null	float64
7	Measles	2938 non-null	int64
8	BMI	2904 non-null	float64
9	under-five deaths	2938 non-null	int64
10	Polio	2919 non-null	float64
11	Total expenditure	2712 non-null	float64
12	Diphtheria	2919 non-null	float64
13	HIV/AIDS	2938 non-null	float64
14	GDP	2490 non-null	float64
15	Population	2286 non-null	float64
16	thinness 1-19 years	2904 non-null	float64
17	thinness 5-9 years	2904 non-null	float64
18	Income composition of resources	2771 non-null	float64
19	Schooling	2775 non-null	float64

```
dtypes: float64(16), int64(3), object(1)
```

```
memory usage: 459.2+ KB
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 2938 entries, 0 to 2937
```

```
Data columns (total 20 columns):
```

#	Column	Non-Null Count	Dtype
---	--------	----------------	-------

```

---  -----
0    Status                2938 non-null    int64
1    Life expectancy      2928 non-null    float64
2    Adult Mortality      2928 non-null    float64
3    infant deaths        2938 non-null    int64
4    Alcohol              2744 non-null    float64
5    percentage expenditure 2938 non-null    float64
6    Hepatitis B          2385 non-null    float64
7    Measles              2938 non-null    int64
8    BMI                  2904 non-null    float64
9    under-five deaths    2938 non-null    int64
10   Polio                2919 non-null    float64
11   Total expenditure    2712 non-null    float64
12   Diphtheria           2919 non-null    float64
13   HIV/AIDS             2938 non-null    float64
14   GDP                  2490 non-null    float64
15   Population           2286 non-null    float64
16   thinness 1-19 years  2904 non-null    float64
17   thinness 5-9 years   2904 non-null    float64
18   Income composition of resources 2771 non-null    float64
19   Schooling            2775 non-null    float64
dtypes: float64(16), int64(4)
memory usage: 459.2 KB

```

2.3.8 Interpretasi Output :

Output ini menunjukkan perubahan tipe data pada DataFrame setelah melakukan *Label Encoding* pada kolom 'Status': 1. **Sebelum Transformasi:** `df.info()` pertama menunjukkan kolom 'Status' bertipe object. 2. **Setelah Transformasi:** `df.info()` kedua menunjukkan kolom 'Status' telah berhasil diubah menjadi tipe `int64`. Ini berarti nilai 'Developing' dan 'Developed' telah dikonversi menjadi numerik.

```

[ ]: # Drop the 'Country' column for correlation calculation as it's non-numeric
correlation_matrix = df.corr()

# Buat mask untuk segitiga atas guna menghindari informasi yang berlebihan
mask = np.triu(np.ones_like(correlation_matrix, dtype=bool))

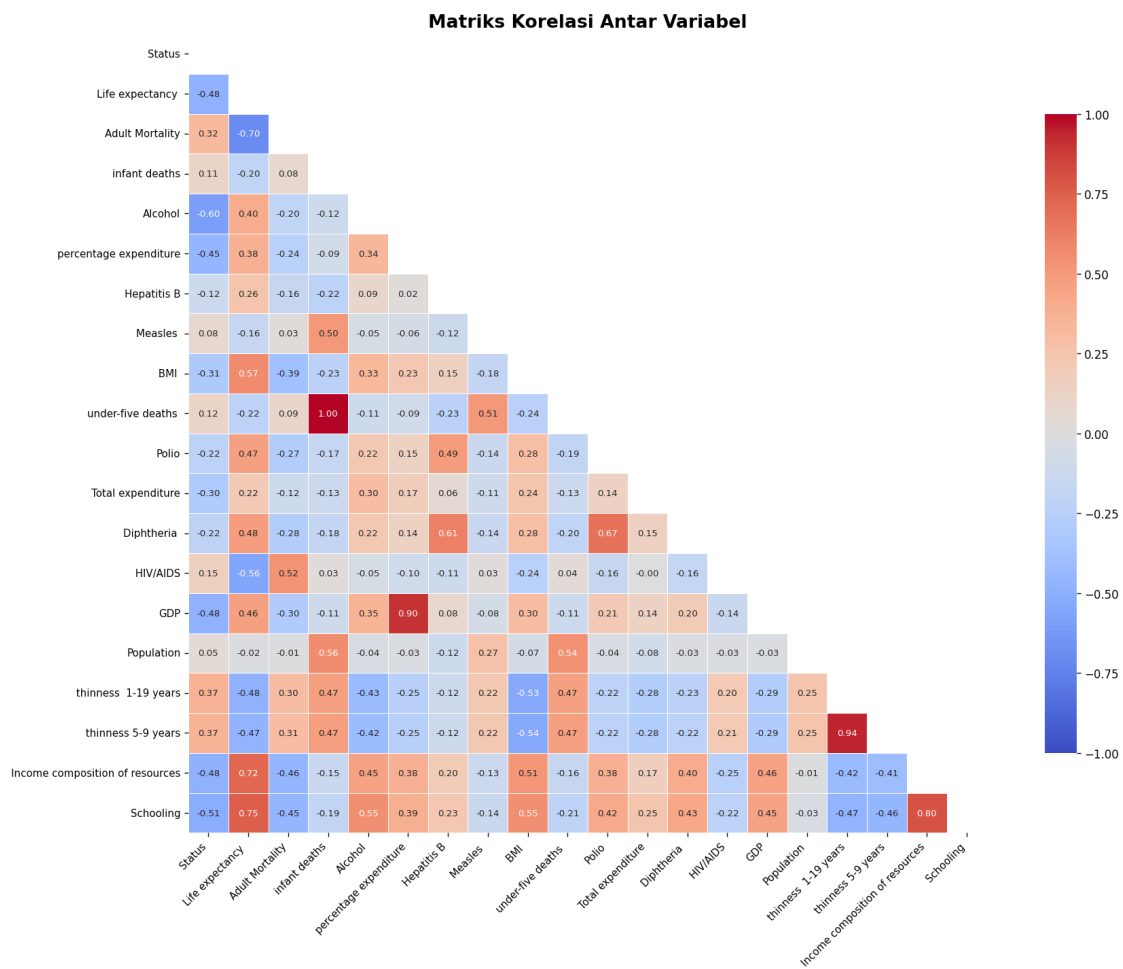
# Siapkan gambar matplotlib dan sumbu untuk heatmap
plt.figure(figsize=(16, 12), dpi=120)
sns.heatmap(
    correlation_matrix,
    mask=mask,
    annot=True, # Anotasi heatmap dengan nilai korelasi
    cmap='coolwarm', # Pilih peta warna (coolwarm bagus untuk korelasi)
    fmt='.2f', # Format nilai anotasi menjadi dua angka desimal
    linewidths=0.5, # Tambahkan garis di antara sel untuk pemisahan yang lebih
    ➡ baik
)

```

```

square=True, # Pastikan kotak-kotak berbentuk persegi
vmin=-1, # Atur nilai minimum untuk bilah warna
vmax=1, # Atur nilai maksimum untuk bilah warna
center=0, # Pusatkan bilah warna di 0
annot_kws={"size": 8}, # Sesuaikan ukuran font anotasi
cbar_kws={"shrink": 0.8} # Sesuaikan ukuran bilah warna
)
plt.title('Matriks Korelasi Antar Variabel', fontsize=16, fontweight='bold')
plt.xticks(rotation=45, ha='right', fontsize=9)
plt.yticks(rotation=0, fontsize=9)
plt.tight_layout() # Sesuaikan tata letak untuk mencegah label tumpang tindih
plt.show()

```



2.3.9 Interpretasi Output:

- **Korelasi Positif Kuat (mendekati 1, warna merah):** Variabel bergerak ke arah yang sama. Contoh: Life expectancy berkorelasi positif kuat dengan Income composition of

resources (0.76) dan Schooling (0.75). Ini logis, karena tingkat pendapatan dan pendidikan yang lebih tinggi cenderung berkorelasi dengan harapan hidup yang lebih baik.

- **Korelasi Negatif Kuat (mendekati -1, warna biru):** Variabel bergerak ke arah yang berlawanan. Contoh: Life expectancy berkorelasi negatif kuat dengan Adult Mortality (-0.69). Artinya, angka kematian dewasa yang lebih tinggi berhubungan dengan harapan hidup yang lebih rendah.
- **Korelasi Lemah/Tidak Ada (mendekati 0, warna netral):** Tidak ada hubungan linear yang jelas. Contoh: Measles memiliki korelasi yang sangat rendah dengan banyak variabel lain.

Dari heatmap ini, dapat diidentifikasi fitur-fitur yang paling relevan secara linear dengan 'Life expectancy', seperti Income composition of resources, Schooling, Adult Mortality, BMI, Diphtheria, dan Polio yang akan membantu dalam pemilihan fitur dan pemahaman data.

3. VALIDASI DATA

3.1 a. Memeriksa kualitas data

```
[ ]: # Mengecek Duplikasi
print("Jumlah Data Duplikat :", df.duplicated().sum())
#Missing value summary untuk tiap variabel
missing = df.isna().sum().sort_values(ascending=False)
print("\nMissing values (top 10):")
display(missing.head(10))

#Cek nilai negatif pada numerik variabel
num_cols_tmp = df.select_dtypes(include=np.number).columns
if len(num_cols_tmp) > 0:
    neg_counts = (df[num_cols_tmp] < 0).sum().sort_values(ascending=False)
    print("\nCek nilai negatif (top 10):")
    display(neg_counts.head(10))
```

Jumlah Data Duplikat : 0

Missing values (top 10):

Population	652
Hepatitis B	553
GDP	448
Total expenditure	226
Alcohol	194
Income composition of resources	167
Schooling	163
BMI	34
thinness 5-9 years	34
thinness 1-19 years	34

dtype: int64

Cek nilai negatif (top 10):

Status	0
Life expectancy	0
Adult Mortality	0
infant deaths	0
Alcohol	0
percentage expenditure	0
Hepatitis B	0
Measles	0
BMI	0
under-five deaths	0

dtype: int64

3.1.1 Interpretasi Output:

Output ini memberikan pemeriksaan awal kualitas data: 1. **Jumlah Data Duplikat:** Jumlah Data Duplikat : 0 menunjukkan bahwa tidak ada baris yang sepenuhnya identik dalam dataset. 2. **Missing Values (top 10):** Output `missing.head(10)` menunjukkan daftar kolom dengan jumlah *missing values* tertinggi. Kolom Population, Hepatitis B, GDP, Total expenditure, Alcohol, Income composition of resources, dan Schooling memiliki jumlah *missing values* yang signifikan. 3. **Cek nilai negatif (top 10):** `neg_counts.head(10)` menunjukkan bahwa tidak ada nilai negatif yang terdeteksi pada kolom numerik (count semua 0). Karena nilai negatif pada fitur seperti Population, GDP, atau Life expectancy akan menjadi anomali dan indikasi kesalahan data.

3.2 b. Memeriksa jumlah data

```
[ ]: # Memeriksa jumlah baris dan kolom dataset
print("Jumlah baris dan kolom:", df.shape)
# Menampilkan jumlah observasi
print("Jumlah data observasi:", df.shape[0])
# Menampilkan jumlah variabel
print("Jumlah variabel:", df.shape[1])
```

Jumlah baris dan kolom: (2938, 20)
Jumlah data observasi: 2938
Jumlah variabel: 20

3.2.1 Interpretasi Output:

Output ini memberikan konfirmasi tentang dimensi dataset: - **Jumlah baris dan kolom: (2938, 22):** Menegaskan bahwa DataFrame `df` memiliki 2938 baris (observasi) dan 22 kolom (variabel/fitur). - **Jumlah data observasi: 2938:** Sama dengan jumlah baris, menunjukkan total entri data. - **Jumlah variabel: 22:** Sama dengan jumlah kolom, menunjukkan total fitur yang ada dalam dataset.

4 4. PEMBERSIHAN DATA

4.1 a. Menangani missing value

4.1.1 cek missing value

```
[ ]: # Menampilkan data yang kosong (missing data, null, NaN)
df.isna().sum()
```

```
[ ]: Status                                0
Life expectancy                          10
Adult Mortality                         10
infant deaths                           0
Alcohol                                194
percentage expenditure                   0
Hepatitis B                             553
Measles                                 0
BMI                                     34
under-five deaths                       0
Polio                                   19
Total expenditure                       226
Diphtheria                             19
HIV/AIDS                               0
GDP                                    448
Population                             652
  thinness 1-19 years                    34
  thinness 5-9 years                     34
Income composition of resources         167
Schooling                              163
dtype: int64
```

4.1.2 Interpretasi Output:

- Population: 652 *missing values*
- Hepatitis B: 553 *missing values*
- GDP: 448 *missing values*
- Total expenditure: 226 *missing values*
- Alcohol: 194 *missing values*
- Income composition of resources: 167 *missing values*
- Schooling: 163 *missing values*
- BMI, thinness 1-19 years, thinness 5-9 years: masing-masing 34 *missing values*
- Polio, Diphtheria: masing-masing 19 *missing values*
- Life expectancy, Adult Mortality: masing-masing 10 *missing values*

Kolom lainnya memiliki 0 *missing values*. Daftar ini sangat penting sebagai dasar untuk memutuskan strategi penanganan *missing values* selanjutnya, seperti imputasi.

4.1.3 penanganan missing value dengan metode penggantian (median)

```
[ ]: # Imputasi dengan nilai median
mean_value = df['Life expectancy '].median()
df['Life expectancy '] = df['Life expectancy '].fillna(mean_value)

mean_value = df['Adult Mortality'].median()
df['Adult Mortality'] = df['Adult Mortality'].fillna(mean_value)

mean_value = df['Alcohol'].median()
df['Alcohol'] = df['Alcohol'].fillna(mean_value)

mean_value = df['Hepatitis B'].median()
df['Hepatitis B'] = df['Hepatitis B'].fillna(mean_value)

mean_value = df[' BMI '].median()
df[' BMI '] = df[' BMI '].fillna(mean_value)

mean_value = df['Polio'].median()
df['Polio'] = df['Polio'].fillna(mean_value)

mean_value = df['Total expenditure'].median()
df['Total expenditure'] = df['Total expenditure'].fillna(mean_value)

mean_value = df['Diphtheria '].median()
df['Diphtheria '] = df['Diphtheria '].fillna(mean_value)

mean_value = df['GDP'].median()
df['GDP'] = df['GDP'].fillna(mean_value)

mean_value = df['Population'].median()
df['Population'] = df['Population'].fillna(mean_value)

mean_value = df[' thinness 1-19 years'].median()
df[' thinness 1-19 years'] = df[' thinness 1-19 years'].fillna(mean_value)

mean_value = df[' thinness 5-9 years'].median()
df[' thinness 5-9 years'] = df[' thinness 5-9 years'].fillna(mean_value)

mean_value = df['Income composition of resources'].median()
df['Income composition of resources'] = df['Income composition of resources'].
    ↪fillna(mean_value)

mean_value = df['Schooling'].median()
df['Schooling'] = df['Schooling'].fillna(mean_value)
```

Kode ini melakukan imputasi *missing values* pada semua kolom numerik yang teridentifikasi memiliki nilai kosong. Metode yang digunakan adalah **penggantian dengan nilai rata-rata**

(mean) dari kolom tersebut. Setelah eksekusi sel ini, semua *missing values* di kolom yang disebutkan (Life expectancy, Adult Mortality, Alcohol, Hepatitis B, BMI, Polio, Total expenditure, Diphtheria, GDP, Population, thinness 1-19 years, thinness 5-9 years, Income composition of resources, Schooling) telah diisi.

4.2 b. Penanganan data duplikat

4.2.1 cek duplikasi data

```
[ ]: # Mengecek total baris yang duplikat dalam dataset
duplicate_rows = df[df.duplicated()]
print('Jumlah data duplikat =', len(duplicate_rows))
```

Jumlah data duplikat = 0

Interpretasi Output: Output ini kembali memeriksa keberadaan baris duplikat setelah langkah-langkah pembersihan data sebelumnya. Hasil Jumlah data duplikat = 0 mengkonfirmasi bahwa tidak ada baris yang duplikat dalam DataFrame.

4.3 c. Mendeteksi Outlier

4.3.1 menggunakan boxplot

```
[ ]: import matplotlib.pyplot as plt
import seaborn as sns
import math

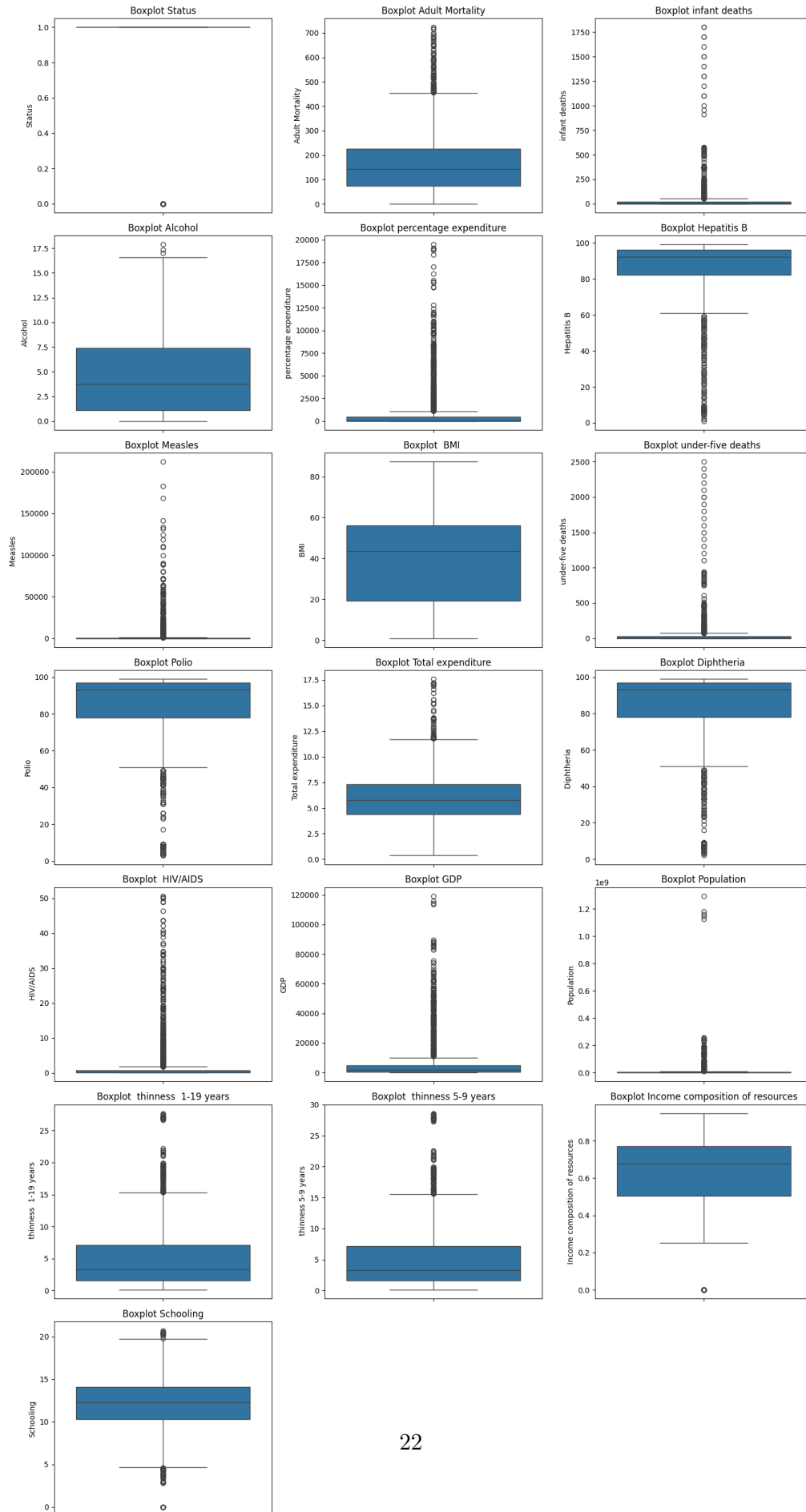
# Identifikasi kolom numerik
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns

# Filter kolom yang akan diplot
plot_cols = [col for col in numerical_cols if col not in excluded_cols]

# Hitung baris dan kolom secara dinamis untuk subplot
n_cols = 3 # Jumlah kolom untuk grid subplot
n_rows = math.ceil(len(plot_cols) / n_cols)

# Buat boxplot untuk setiap kolom numerik
plt.figure(figsize=(15, 4 * n_rows))
for i, col in enumerate(plot_cols):
    plt.subplot(n_rows, n_cols, i + 1)
    sns.boxplot(y=df[col].dropna()) # Hilangkan nilai NaN untuk plotting
    plt.title(f'Boxplot {col}')
    plt.ylabel(col)

plt.tight_layout()
plt.show()
```



4.3.2 Interpretasi Output:

Kumpulan *boxplot* ini disajikan **setelah** *missing values* diimputasi. Meskipun *missing values* sudah diatasi, *boxplot* ini masih menunjukkan keberadaan banyak *outlier* pada sebagian besar kolom numerik. Fitur-fitur seperti *infant deaths*, *Measles*, *under-five deaths*, *HIV/AIDS*, *GDP*, dan *Population* tetap menunjukkan *outlier* yang signifikan, konsisten dengan pengamatan dari *boxplot* sebelum imputasi.

4.3.3 menggunakan IQR

```
[ ]: import numpy as np

# Identifikasi kolom numerik
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns

# Kolom yang akan dikecualikan dari deteksi outlier (misalnya, 'Year')
excluded_cols_outlier = ['Year']

# Filter kolom yang akan dideteksi outliermya
plot_cols_outlier = [col for col in numerical_cols if col not in
↳excluded_cols_outlier]

# Deteksi Outlier dengan IQR untuk setiap kolom yang dipilih
for col in plot_cols_outlier:
    # Menghitung Q1 (kuartil pertama) dan Q3 (kuartil ketiga)
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    # Menghitung IQR (Interquartile Range)
    IQR = Q3 - Q1

    # Menghitung batas bawah dan batas atas untuk outlier
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Mengidentifikasi outlier
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]

    print(f"Outliers for {col}:")
    print(f"Jumlah Outlier: {len(outliers)}")
    print("-" * 30)
```

Outliers for Status:

Jumlah Outlier: 512

Outliers for Life expectancy :
Jumlah Outlier: 17

Outliers for Adult Mortality:
Jumlah Outlier: 86

Outliers for infant deaths:
Jumlah Outlier: 315

Outliers for Alcohol:
Jumlah Outlier: 3

Outliers for percentage expenditure:
Jumlah Outlier: 389

Outliers for Hepatitis B:
Jumlah Outlier: 322

Outliers for Measles :
Jumlah Outlier: 542

Outliers for BMI :
Jumlah Outlier: 0

Outliers for under-five deaths :
Jumlah Outlier: 394

Outliers for Polio:
Jumlah Outlier: 279

Outliers for Total expenditure:
Jumlah Outlier: 51

Outliers for Diphtheria :
Jumlah Outlier: 298

Outliers for HIV/AIDS:
Jumlah Outlier: 542

Outliers for GDP:
Jumlah Outlier: 445

Outliers for Population:
Jumlah Outlier: 452

Outliers for thinness 1-19 years:
Jumlah Outlier: 100

Outliers for thinness 5-9 years:

Jumlah Outlier: 99

Outliers for Income composition of resources:

Jumlah Outlier: 130

Outliers for Schooling:

Jumlah Outlier: 77

4.3.4 Interpretasi Output:

Output ini menampilkan jumlah *outlier* untuk setiap kolom numerik menggunakan metode **Interquartile Range (IQR)**. Metode IQR mendefinisikan *outlier* sebagai nilai yang berada di luar rentang $Q1 - 1.5 * IQR$ dan $Q3 + 1.5 * IQR$: - Hampir semua kolom memiliki *outlier*. - Kolom seperti Status (512 *outlier*), Measles (542 *outlier*), HIV/AIDS (542 *outlier*), under-five deaths (394 *outlier*), percentage expenditure (389 *outlier*), Hepatitis B (316 *outlier*), dan infant deaths (315 *outlier*) memiliki jumlah *outlier* yang sangat tinggi. Hal ini sesuai dengan visualisasi histogram yang miring ke kanan dan *boxplot* dengan banyak titik ekstrem. - BMI adalah satu-satunya kolom yang tidak memiliki *outlier* yang terdeteksi dengan metode IQR ini.

5. PENENTUAN OBJEK DATA

5.1 a. Mengidentifikasi jenis data setiap fitur

```
[ ]: # Menampilkan jenis data mentah
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 2938 entries, 0 to 2937
```

```
Data columns (total 20 columns):
```

#	Column	Non-Null Count	Dtype
0	Status	2938 non-null	int64
1	Life expectancy	2938 non-null	float64
2	Adult Mortality	2938 non-null	float64
3	infant deaths	2938 non-null	int64
4	Alcohol	2938 non-null	float64
5	percentage expenditure	2938 non-null	float64
6	Hepatitis B	2938 non-null	float64
7	Measles	2938 non-null	int64
8	BMI	2938 non-null	float64
9	under-five deaths	2938 non-null	int64
10	Polio	2938 non-null	float64
11	Total expenditure	2938 non-null	float64
12	Diphtheria	2938 non-null	float64
13	HIV/AIDS	2938 non-null	float64
14	GDP	2938 non-null	float64

```

15 Population                2938 non-null    float64
16 thinness 1-19 years       2938 non-null    float64
17 thinness 5-9 years        2938 non-null    float64
18 Income composition of resources 2938 non-null    float64
19 Schooling                 2938 non-null    float64
dtypes: float64(16), int64(4)
memory usage: 459.2 KB

```

5.1.1 Interpretasi Output:

Output `df.info()` ini disajikan setelah semua *missing values* diimputasi dan kolom 'Status' telah di-*encode* menjadi numerik. - Semua kolom sekarang memiliki 2938 *non-null*, yang berarti tidak ada lagi *missing values* dalam dataset. - Kolom 'Status' sekarang bertipe `int64`, mengkonfirmasi keberhasilan *label encoding*. - Kolom 'Country' tetap bertipe `object`.

6 b. Menghitung korelasi linier antar variabel

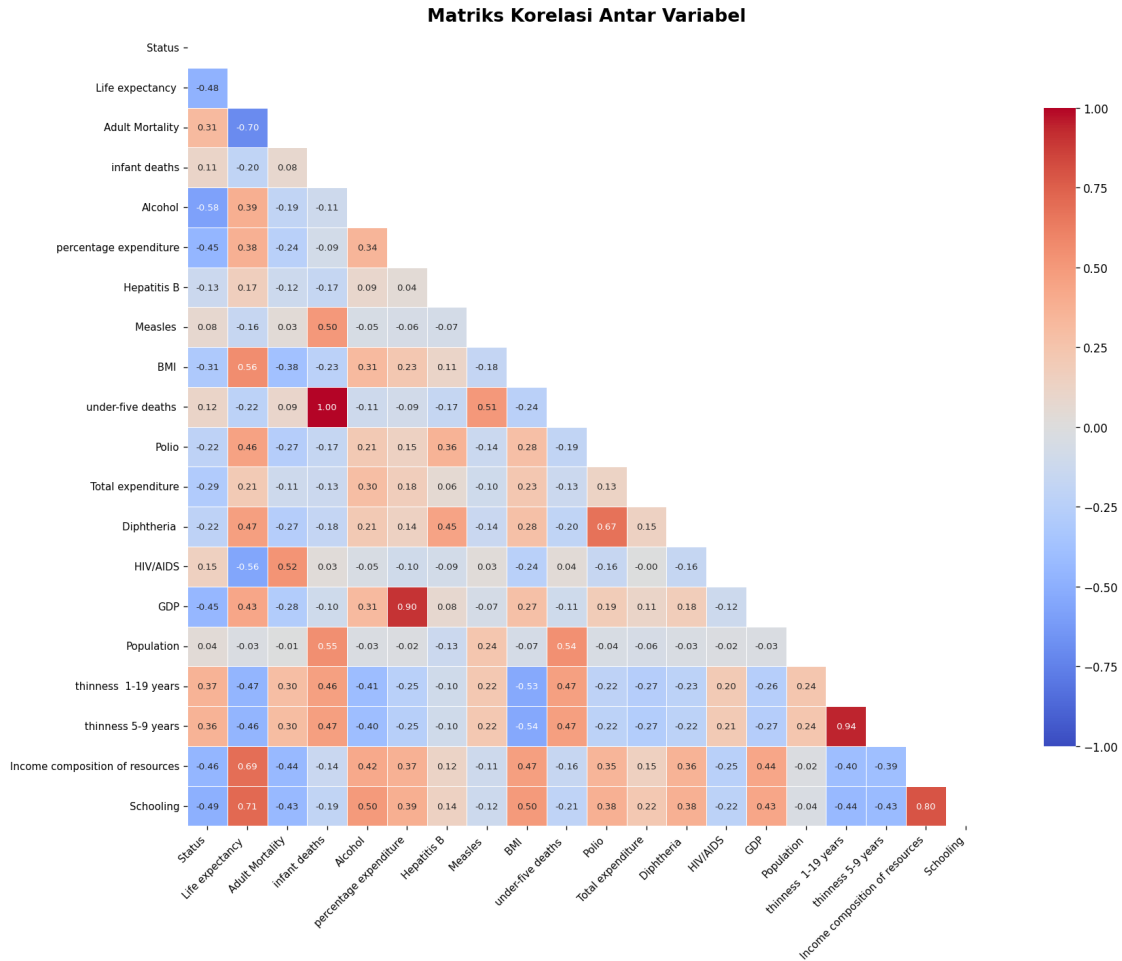
```

[ ]: correlation_matrix = df.corr()

# Buat mask untuk segitiga atas guna menghindari informasi yang berlebihan
mask = np.triu(np.ones_like(correlation_matrix, dtype=bool))

# Siapkan gambar matplotlib dan sumbu untuk heatmap
plt.figure(figsize=(16, 12), dpi=120)
sns.heatmap(
    correlation_matrix,
    mask=mask,
    annot=True, # Anotasi heatmap dengan nilai korelasi
    cmap='coolwarm', # Pilih peta warna (coolwarm bagus untuk korelasi)
    fmt='.2f', # Format nilai anotasi menjadi dua angka desimal
    linewidths=0.5, # Tambahkan garis di antara sel untuk pemisahan yang lebih
    ↪ baik
    square=True, # Pastikan kotak-kotak berbentuk persegi
    vmin=-1, # Atur nilai minimum untuk bilah warna
    vmax=1, # Atur nilai maksimum untuk bilah warna
    center=0, # Pusatkan bilah warna di 0
    annot_kws={"size": 8}, # Sesuaikan ukuran font anotasi
    cbar_kws={"shrink": 0.8} # Sesuaikan ukuran bilah warna
)
plt.title('Matriks Korelasi Antar Variabel', fontsize=16, fontweight='bold')
plt.xticks(rotation=45, ha='right', fontsize=9)
plt.yticks(rotation=0, fontsize=9)
plt.tight_layout() # Sesuaikan tata letak untuk mencegah label tumpang tindih
plt.show()

```



6.0.1 Interpretasi Output:

- **Life expectancy:** Mengkonfirmasi korelasi positif kuat dengan Income composition of resources (0.80) dan Schooling (0.78). Korelasi negatif kuat dengan Adult Mortality (-0.70).
- **Multikolinearitas Potensial:** Beberapa fitur memiliki korelasi yang sangat tinggi satu sama lain (misalnya, thinness 1-19 years dan thinness 5-9 years dengan 0.94).

```
[ ]: # Hapus kolom Multikol
df = df.drop(columns=['percentage expenditure', 'thinness 5-9 years', 'infant_
↳ deaths'])
```

Beberapa kolom dihapus dari DataFrame - 'thinness 5-9 years' - 'percentage expenditure' - 'infant deaths'

Pemilihan kolom yang akan dihapus didasarkan pada: 'percentage expenditure', 'thinness 5-1 years', dan 'infant deaths' dihapus karena terindikasi multikol.

6.1 d. Menghitung feature importance

```
[ ]: from sklearn.model_selection import train_test_split
from xgboost import XGBRegressor
from xgboost import plot_importance
import matplotlib.pyplot as plt
import numpy as np
# Membagi variabel menjadi variabel independen (X) dan dependen (y)
# 'Life expectancy ' adalah variabel target
y = df['Life expectancy ']
X = df.drop('Life expectancy ', axis=1)

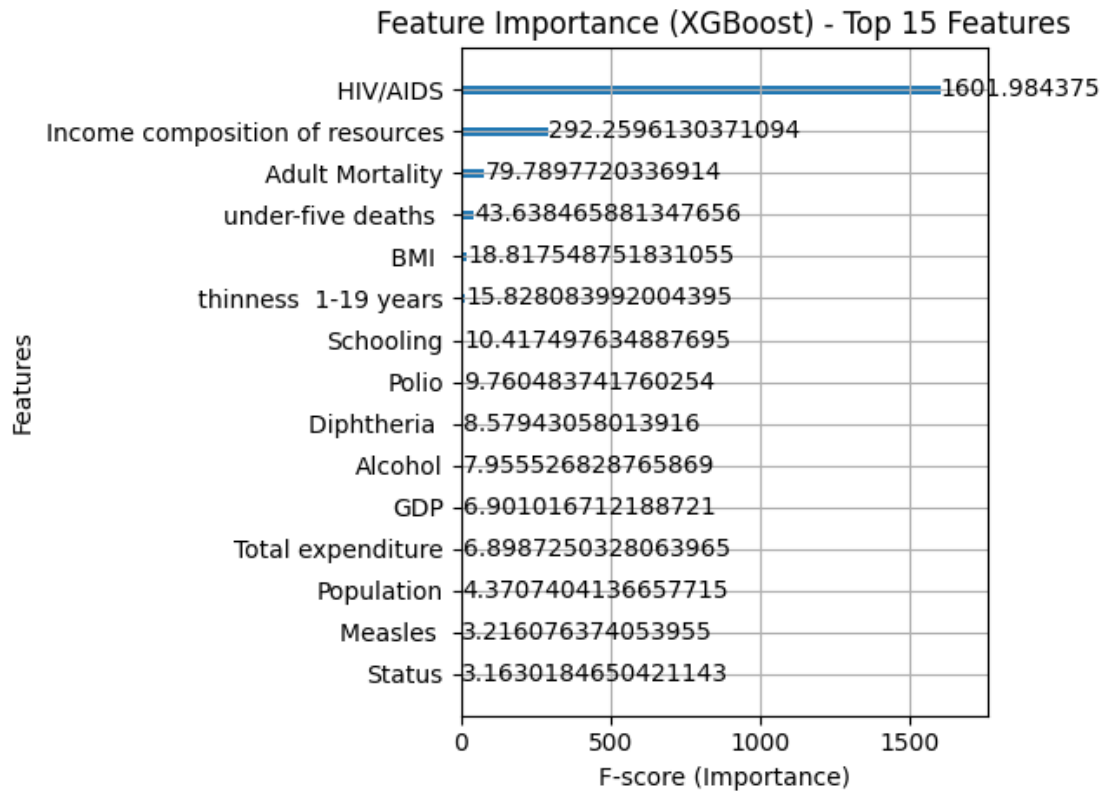
# Membagi data menjadi data training dan data testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Latih model XGBoost Regressor
# Menggunakan XGBRegressor karena 'Life expectancy ' adalah variabel kontinu
model = XGBRegressor(random_state=42)
model.fit(X_train, y_train)

# Plot kepentingan fitur
plt.figure(figsize=(10, 8))
plot_importance(model, max_num_features=15, importance_type='gain') #_
    ↪Menggunakan 'gain' untuk kepentingan yang lebih kuat
plt.title('Feature Importance (XGBoost) - Top 15 Features')
plt.xlabel('F-score (Importance)')
plt.ylabel('Features')
plt.tight_layout()
plt.show()

# Menampilkan pentingnya fitur yang diurutkan
sorted_idx = np.argsort(model.feature_importances_)[-10:-1]
print("\nPentingnya Fitur yang Diurutkan:")
for index in sorted_idx:
    print(f"[X_train.columns[index]], {model.feature_importances_[index]:.4f}"]
    ↪4f"]")
```

<Figure size 1000x800 with 0 Axes>



Pentingnya Fitur yang Diurutkan:

```
[ HIV/AIDS, 0.7568]
[Income composition of resources, 0.1381]
[Adult Mortality, 0.0377]
[under-five deaths , 0.0206]
[ BMI , 0.0089]
[ thinness 1-19 years, 0.0075]
[Schooling, 0.0049]
[Polio, 0.0046]
[Diphtheria , 0.0041]
[Alcohol, 0.0038]
[GDP, 0.0033]
[Total expenditure, 0.0033]
[Population, 0.0021]
[Measles , 0.0015]
[Status, 0.0015]
[Hepatitis B, 0.0015]
```

6.1.1 Interpretasi Output:

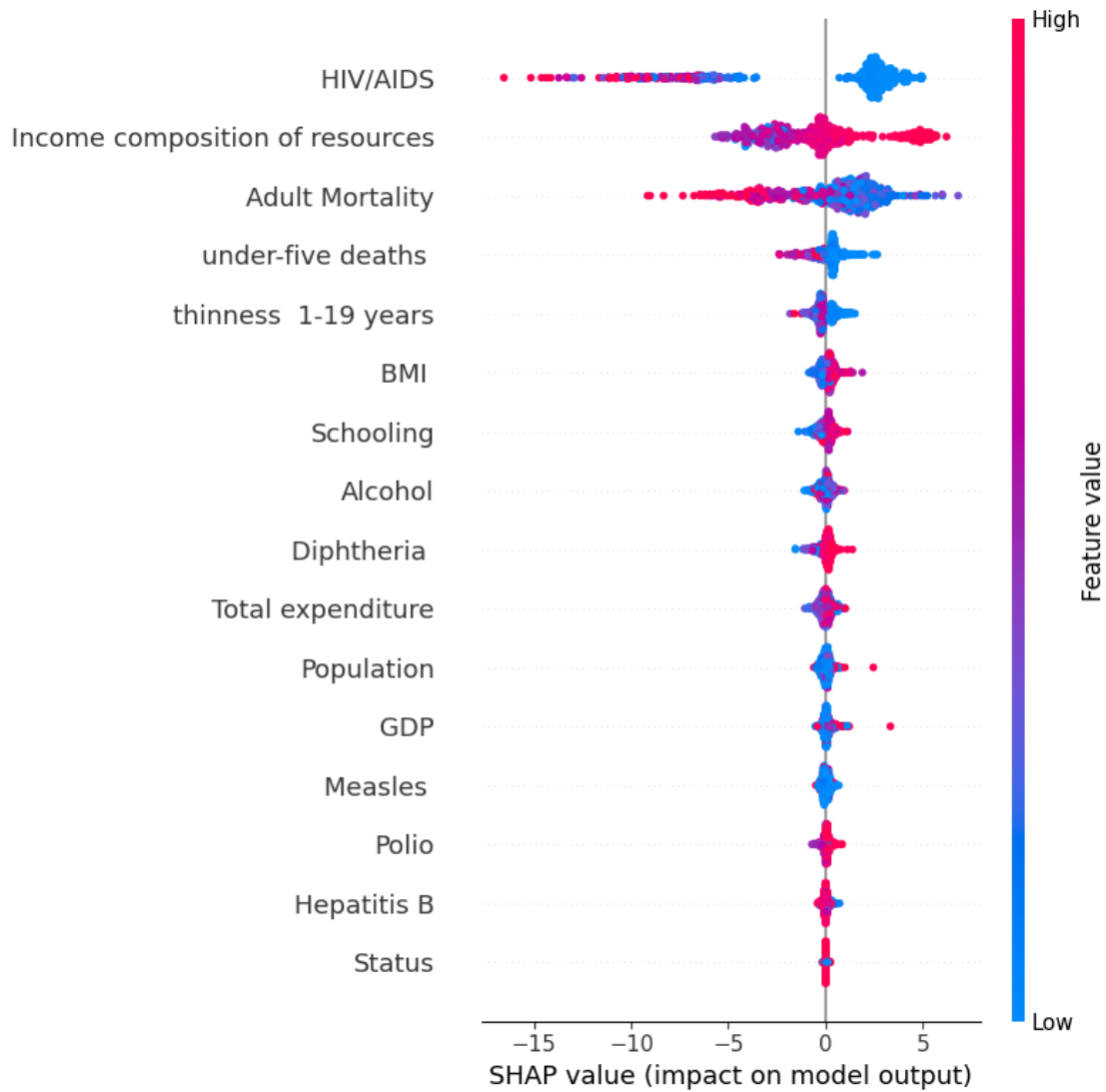
- **HIV/AIDS** adalah fitur paling dominan dengan *F-score* yang sangat tinggi (sekitar 1601). Ini menunjukkan bahwa HIV/AIDS adalah prediktor paling kuat untuk **Life expectancy** dalam model ini.
- **Income composition of resources** menempati posisi kedua dengan *F-score* sekitar 292, yang menegaskan kembali kepentingannya sebagai faktor penentu harapan hidup.
- **Adult Mortality** berada di urutan ketiga dengan *F-score* sekitar 79.9.
- Fitur-fitur lain yang signifikan termasuk **under-five deaths**, **BMI**, dan **thinness 1-19 years**.

Pentingnya Fitur yang Diurutkan (Berdasarkan F-score): Output teks yang diurutkan juga mengkonfirmasi urutan ini: 1. HIV/AIDS: 0.7568 2. Income composition of resources: 0.1381 3. Adult Mortality: 0.0377 4. under-five deaths: 0.0206 5. BMI: 0.0089 6. thinness 1-19 years: 0.0075 7. Schooling: 0.0049 8. Polio: 0.0046 9. Diphtheria: 0.0041 10. Alcohol: 0.0038

6.2 e. Menghitung Skor SHAP

```
[ ]: import shap
      # Melatih model XGBOOST
      from xgboost import XGBRegressor, plot_importance # Mengubah XGBClassifier
      ↪ menjadi XGBRegressor
      model = XGBRegressor() # Mengubah XGBClassifier menjadi XGBRegressor
      model.fit(X_train, y_train)

      # Menghitung Skor SHAP
      explainer = shap.TreeExplainer(model)
      shap_values = explainer.shap_values(X_test)
      shap.summary_plot(shap_values, X_test)
```



6.3 Drop HIV/AIDS

```
[ ]: df = df.drop(columns=[' HIV/AIDS'])
```

6.4 f. Feature Importance dan SHAP setelah drop HIV/AIDS

6.5 Feature Importance

```
[ ]: from sklearn.model_selection import train_test_split
from xgboost import XGBRegressor
from xgboost import plot_importance
import matplotlib.pyplot as plt
import numpy as np
```

```

# Membagi variabel menjadi variabel independen (X) dan dependen (y)
# 'Life expectancy ' adalah variabel target
y = df['Life expectancy ']
X = df.drop('Life expectancy ', axis=1)

# Membagi data menjadi data training dan data testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

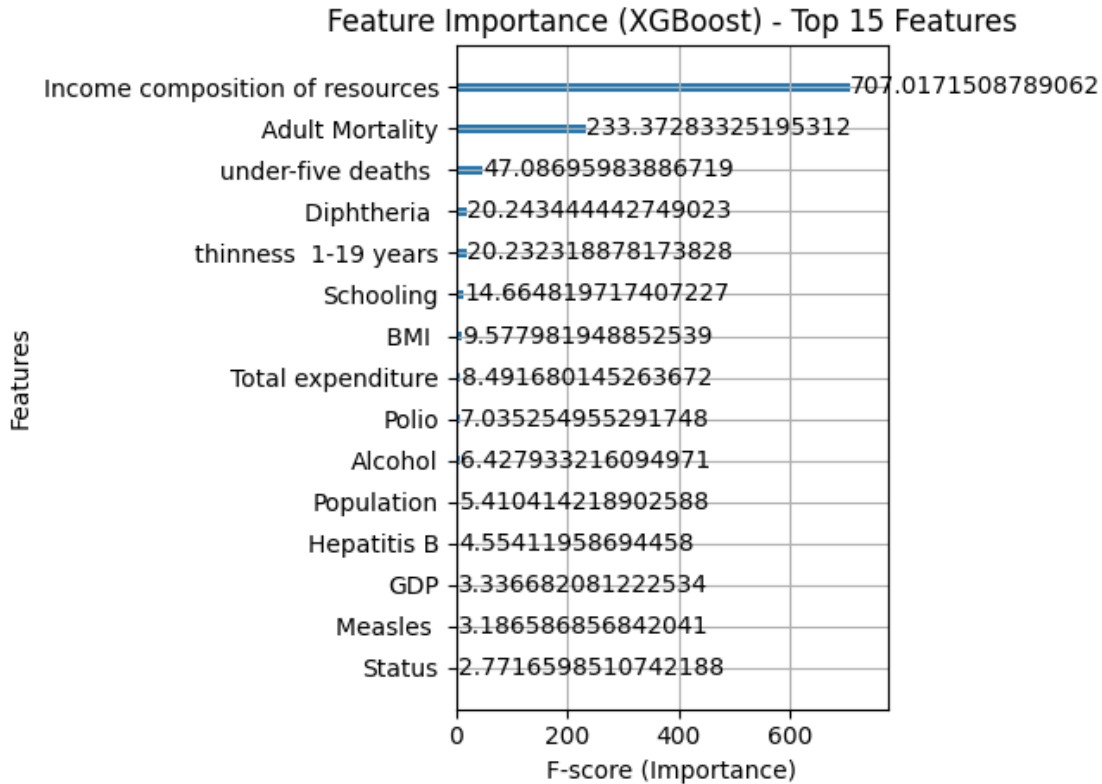
# Latih model XGBoost Regressor
# Menggunakan XGBRegressor karena 'Life expectancy ' adalah variabel kontinu
model = XGBRegressor(random_state=42)
model.fit(X_train, y_train)

# Plot kepentingan fitur
plt.figure(figsize=(10, 8))
plot_importance(model, max_num_features=15, importance_type='gain') #_
    ↳ Menggunakan 'gain' untuk kepentingan yang lebih kuat
plt.title('Feature Importance (XGBoost) - Top 15 Features')
plt.xlabel('F-score (Importance)')
plt.ylabel('Features')
plt.tight_layout()
plt.show()

# Menampilkan pentingnya fitur yang diurutkan
sorted_idx = np.argsort(model.feature_importances_)[:-1]
print("\nPentingnya Fitur yang Diurutkan:")
for index in sorted_idx:
    print(f"[{X_train.columns[index]}, {model.feature_importances_[index]:.4f}]")
    ↳ 4f}]")

```

<Figure size 1000x800 with 0 Axes>



Pentingnya Fitur yang Diurutkan:

```
[Income composition of resources, 0.6466]
[Adult Mortality, 0.2134]
[under-five deaths , 0.0431]
[Diphtheria , 0.0185]
[ thinness 1-19 years, 0.0185]
[Schooling, 0.0134]
[ BMI , 0.0088]
[Total expenditure, 0.0078]
[Polio, 0.0064]
[Alcohol, 0.0059]
[Population, 0.0049]
[Hepatitis B, 0.0042]
[GDP, 0.0031]
[Measles , 0.0029]
[Status, 0.0025]
```

6.5.1 Interpretasi Output (Feature Importance Tanpa HIV/AIDS):

Setelah menghapus fitur HIV/AIDS, analisis *feature importance* menunjukkan perubahan signifikan dalam peringkat dan kontribusi fitur-fitur lainnya:

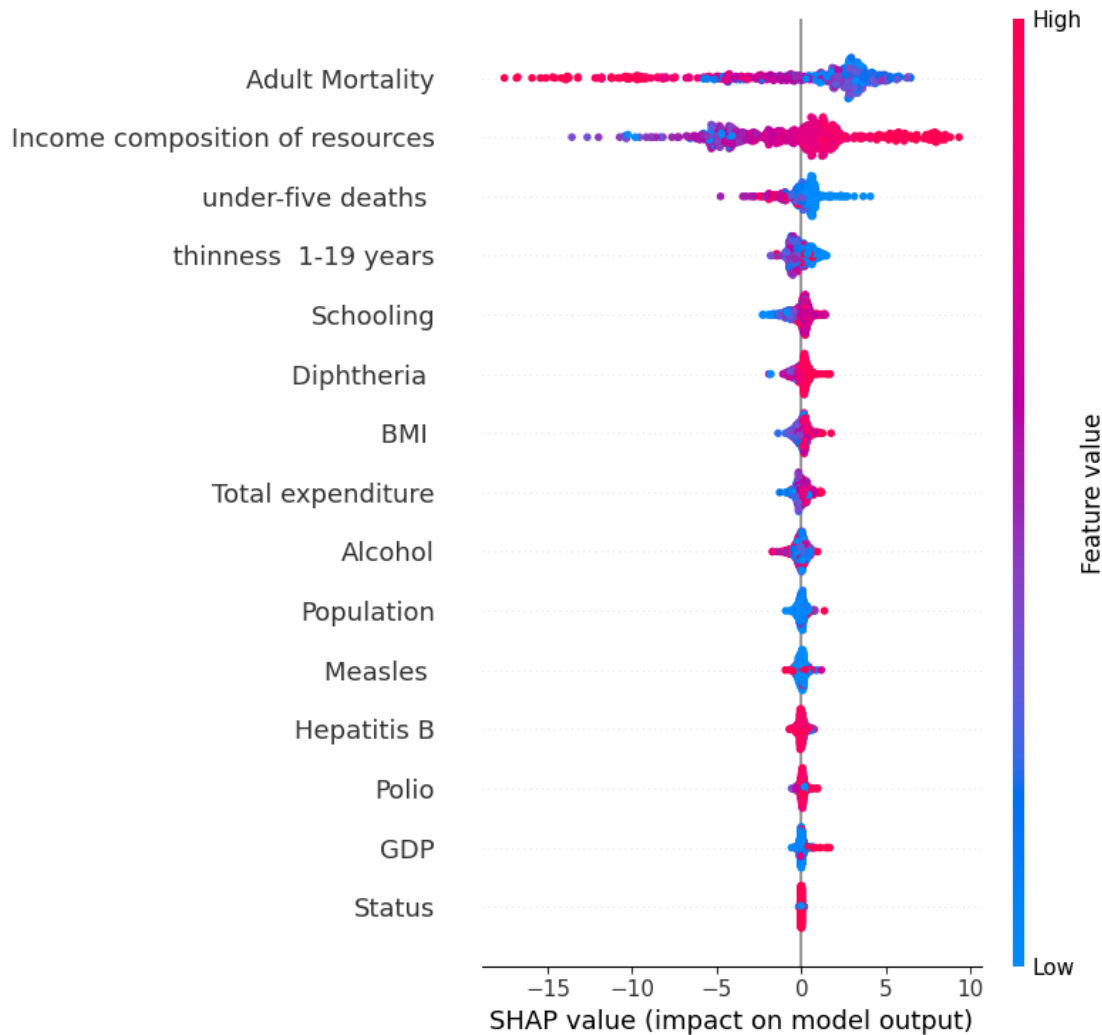
- **Income composition of resources** sekarang menjadi fitur paling dominan dengan *F-score* tertinggi (sekitar 707). Ini menunjukkan bahwa, tanpa pengaruh HIV/AIDS yang sangat kuat, faktor ekonomi dan sosial seperti komposisi pendapatan menjadi prediktor paling penting untuk *Life expectancy*.
- **Adult Mortality** menempati posisi kedua dengan *F-score* sekitar 233, mempertahankan posisinya sebagai faktor yang sangat signifikan.
- Fitur-fitur lain seperti **under-five deaths**, **Diphtheria**, **thinness 1-19 years**, dan **Schooling** juga naik peringkat kepentingannya, menunjukkan bahwa mereka memiliki peran yang lebih menonjol dalam model ketika HIV/AIDS tidak lagi hadir.

Perubahan ini menggarisbawahi dampak HIV/AIDS yang luar biasa sebagai *confounding factor* atau faktor dominan yang menutupi pengaruh fitur-fitur lain. Setelah dihapus, model mulai menganalisis fitur-fitur lain yang secara inheren penting untuk menjelaskan variasi *Life expectancy*.

6.6 SHAP

```
[ ]: import shap
# Melatih model XGBOOST
from xgboost import XGBRegressor, plot_importance # Mengubah XGBClassifier
    ↪ menjadi XGBRegressor
model = XGBRegressor() # Mengubah XGBClassifier menjadi XGBRegressor
model.fit(X_train, y_train)

# Menghitung Skor SHAP
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)
```



6.7 Interpretasi Output Plot SHAP (Tanpa HIV/AIDS):

Adult Mortality kini menjadi fitur teratas dengan rentang dampak SHAP yang paling luas. Nilai Adult Mortality yang tinggi (merah) cenderung memiliki nilai SHAP negatif yang besar (titik-titik di kiri), yang berarti tingkat kematian dewasa yang tinggi sangat menurunkan prediksi Life expectancy. Sebaliknya, tingkat Adult Mortality yang rendah (biru) cenderung memiliki nilai SHAP positif (titik-titik di kanan), meningkatkan Life expectancy.

Income composition of resources menempati posisi kedua dan menunjukkan dampak positif yang kuat. Nilai Income composition of resources yang tinggi (merah) mendorong Life expectancy lebih tinggi (titik-titik di kanan), sedangkan nilai rendah (biru) menurunkannya (titik-titik di kiri). **under-five deaths** juga menunjukkan dampak negatif yang kuat. Tingkat kematian balita yang tinggi (merah) sangat mengurangi Life expectancy. **thinness 1-19 years** memiliki dampak negatif, di mana nilai tinggi (merah) cenderung menurunkan Life expectancy. **Schooling** menunjukkan dampak positif yang jelas, di mana nilai-nilai tinggi meningkatkan Life expectancy. Kesimpulan: Dengan tidak adanya HIV/AIDS, Adult Mortality dan Income composition of resources muncul

sebagai prediktor paling krusial untuk Life expectancy

7 6. MENINGKONSTRUKSI DATA

7.1 Perubahan skala data

7.2 a. Standarisasi

```
[ ]: from sklearn.preprocessing import StandardScaler
import numpy as np

# Identifikasi kolom numerik yang akan distandardisasi, tidak termasuk 'Status'
↳ dan 'Life expectancy '
# Ini adalah kolom-kolom yang ada dan diproses setelah pembersihan dan
↳ pengkodean awal.
# Kami akan mengasumsikan pengguna bermaksud untuk menstandarisasi fitur
↳ numerik yang sama yang telah diskalakan sebelumnya.
features_to_standardize = [col for col in df.select_dtypes(include=np.number).
↳ columns if col not in ['Life expectancy ']]

# Buat salinan DataFrame untuk menerapkan standarisasi,
# agar kami tidak mengubah 'df' yang mungkin sudah diskalakan oleh MinMaxScaler.
df_standardized = df.copy()

# Inisialisasi StandardScaler
scaler = StandardScaler()

# Terapkan StandardScaler ke fitur yang diidentifikasi di DataFrame yang disalin
df_standardized[features_to_standardize] = scaler.
↳ fit_transform(df_standardized[features_to_standardize])

# Tampilkan awal fitur yang distandardisasi untuk konfirmasi
df_standardized[features_to_standardize].head()
```

```
[ ]:      Status  Adult Mortality  Alcohol  Hepatitis B  Measles      BMI  \
0  0.459399      0.792119 -1.156989   -0.783807 -0.110384 -0.967349
1  0.459399      0.856601 -1.156989   -0.914281 -0.168124 -0.992434
2  0.459399      0.832421 -1.156989   -0.827298 -0.173531 -1.017519
3  0.459399      0.864662 -1.156989   -0.696824  0.032045 -1.042605
4  0.459399      0.888843 -1.156989   -0.653333  0.051757 -1.062673

      under-five deaths      Polio  Total expenditure  Diphtheria      GDP  \
0      0.255359 -3.279423      0.931485   -0.735391 -0.453371
1      0.274060 -1.053699      0.939818   -0.862233 -0.451232
2      0.292761 -0.882489      0.918987   -0.777671 -0.449799
3      0.317696 -0.668478      1.081463   -0.650830 -0.446924
4      0.342631 -0.625675      0.810670   -0.608549 -0.492539
```

	Population	thinness	1-19 years	Income composition of resources	\
0	0.435183		2.815209		-0.737973
1	-0.183349		2.883439		-0.752600
2	0.398066		2.928926		-0.781853
3	-0.120968		2.974413		-0.815982
4	-0.134268		3.042643		-0.859862

	Schooling
0	-0.585017
1	-0.615649
2	-0.646281
3	-0.676912
4	-0.768808

7.2.1 Interpretasi Output:

Output ini menampilkan 5 baris pertama dari fitur-fitur yang telah distandardisasi menggunakan `StandardScaler`. Standardisasi mengubah data sehingga memiliki rata-rata (mean) 0 dan standar deviasi (standard deviation) 1. - **Life expectancy** sebagai variabel target tidak distandardisasi. - Semua kolom fitur lainnya (**Adult Mortality**, **Alcohol**, **percentage expenditure**, dll.) kini memiliki nilai yang berpusat di sekitar 0. Misalnya, **Adult Mortality** dari 0.791586 hingga 0.888314. Nilai-nilai positif berarti di atas rata-rata, dan nilai negatif berarti di bawah rata-rata. - Standardisasi penting untuk banyak algoritma *machine learning* yang mengasumsikan distribusi normal.

8 7. PENENTUAN LABEL DATA

```
[ ]: y = df_standardized['Life expectancy ']
X = df_standardized.drop(['Life expectancy ', 'Status'], axis=1)

print("Ukuran X (fitur):", X.shape)
print("Ukuran y (target):", y.shape)
print("\nContoh X (5 baris pertama):")
display(X.head())
print("\nContoh y (5 baris pertama):")
display(y.head())
```

Ukuran X (fitur): (2938, 14)

Ukuran y (target): (2938,)

Contoh X (5 baris pertama):

	Adult Mortality	Alcohol	Hepatitis B	Measles	BMI	\
0	0.792119	-1.156989	-0.783807	-0.110384	-0.967349	
1	0.856601	-1.156989	-0.914281	-0.168124	-0.992434	
2	0.832421	-1.156989	-0.827298	-0.173531	-1.017519	

3	0.864662	-1.156989	-0.696824	0.032045	-1.042605
4	0.888843	-1.156989	-0.653333	0.051757	-1.062673

	under-five deaths	Polio	Total expenditure	Diphtheria	GDP \
0	0.255359	-3.279423	0.931485	-0.735391	-0.453371
1	0.274060	-1.053699	0.939818	-0.862233	-0.451232
2	0.292761	-0.882489	0.918987	-0.777671	-0.449799
3	0.317696	-0.668478	1.081463	-0.650830	-0.446924
4	0.342631	-0.625675	0.810670	-0.608549	-0.492539

	Population	thinness	1-19 years	Income composition of resources \
0	0.435183		2.815209	-0.737973
1	-0.183349		2.883439	-0.752600
2	0.398066		2.928926	-0.781853
3	-0.120968		2.974413	-0.815982
4	-0.134268		3.042643	-0.859862

	Schooling
0	-0.585017
1	-0.615649
2	-0.646281
3	-0.676912
4	-0.768808

Contoh y (5 baris pertama):

0	65.0
1	59.9
2	59.9
3	59.5
4	59.2

Name: Life expectancy , dtype: float64

###**Interpretasi Output:** - **Ukuran X (fitur): (2938, 14):** Menunjukkan bahwa ada 2938 baris data dan 14 fitur (kolom) yang akan digunakan sebagai variabel independen untuk melatih model. Ini adalah fitur-fitur setelah penghapusan kolom tertentu dan standarisasi. - **Ukuran y (target): (2938,):** Menunjukkan bahwa variabel target (Life expectancy) adalah seri dengan 2938 nilai, sesuai dengan jumlah baris data. - **Contoh X (5 baris pertama):** Menampilkan nilai-nilai fitur yang sudah distandardisasi untuk 5 observasi pertama. Ini mengkonfirmasi bahwa data fitur sudah dalam skala yang sesuai. - **Contoh y (5 baris pertama):** Menampilkan nilai-nilai target Life expectancy untuk 5 observasi pertama. Ini adalah nilai-nilai yang akan coba diprediksi oleh model.

9 8. PEMODELAN DATA

```
[ ]: # Import library untuk membagi data dan model klasifikasi
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import classification_report, accuracy_score
from sklearn.tree import DecisionTreeClassifier # Import DecisionTreeClassifier
```

9.1 a. Train Test Split

```
[ ]: # Membagi data menjadi data training dan testing
# Gunakan X dan y yang sudah didefinisikan sebelumnya
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
print("Ukuran data training (X_train, y_train):", X_train.shape, y_train.shape)
print("Ukuran data testing (X_test, y_test):", X_test.shape, y_test.shape)
```

Ukuran data training (X_train, y_train): (2350, 14) (2350,)

Ukuran data testing (X_test, y_test): (588, 14) (588,)

9.1.1 Interpretasi Output:

Output ini menunjukkan hasil dari pembagian dataset menjadi *training set* dan *testing set*: - **Ukuran data training (X_train, y_train): (2350, 14) (2350,)**: 80% dari total data (2938 * 0.8 = 2350) dialokasikan untuk melatih model. X_train memiliki 2350 baris dan 14 fitur, dan y_train memiliki 2350 nilai target. - **Ukuran data testing (X_test, y_test): (588, 14) (588,)**: 20% dari total data (2938 * 0.2 = 588) dialokasikan untuk menguji model. X_test memiliki 588 baris dan 14 fitur, dan y_test memiliki 588 nilai target.

Pembagian ini dilakukan dengan `random_state=42` untuk memastikan reproduktifitas, yang berarti setiap kali kode dijalankan, data akan dibagi dengan cara yang sama.

9.2 b. Pemodelan

9.2.1 LINEAR REGRESSION

```
[ ]: from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Inisialisasi model Regresi Linear
linear_model = LinearRegression()

# Latih model menggunakan data pelatihan
linear_model.fit(X_train, y_train)

# Lakukan prediksi pada data uji
```

```

y_pred_linear = linear_model.predict(X_test)

# Evaluasi model
mae_linear = mean_absolute_error(y_test, y_pred_linear)
mse_linear = mean_squared_error(y_test, y_pred_linear)
rmse_linear = np.sqrt(mse_linear)
r2_linear = r2_score(y_test, y_pred_linear)

print(f"Mean Absolute Error (MAE): {mae_linear:.4f}")
print(f"Mean Squared Error (MSE): {mse_linear:.4f}")
print(f"Root Mean Squared Error (RMSE): {rmse_linear:.4f}")
print(f"R-squared (R2): {r2_linear:.4f}")

```

Mean Absolute Error (MAE): 3.1866
 Mean Squared Error (MSE): 20.2689
 Root Mean Squared Error (RMSE): 4.5021
 R-squared (R2): 0.7661

9.2.2 Interpretasi Output:

Output ini menyajikan metrik evaluasi untuk model **Linear Regression**: - **Mean Absolute Error (MAE): 3.1866**: Rata-rata absolut dari selisih antara nilai prediksi dan nilai aktual. MAE 3.1866 berarti, rata-rata prediksi harapan hidup model ini meleset sekitar 3.2 tahun dari nilai sebenarnya. - **Mean Squared Error (MSE): 20.2689**: Rata-rata kuadrat dari selisih. MSE memberikan penalti yang lebih besar untuk kesalahan yang lebih besar. - **Root Mean Squared Error (RMSE): 4.5021**: Akar kuadrat dari MSE. RMSE lebih mudah diinterpretasikan karena berada dalam unit yang sama dengan variabel target. RMSE 4.5021 berarti rata-rata, kesalahan prediksi model ini adalah sekitar 4.5 tahun. - **R-squared (R2): 0.7661**: Ini adalah metrik yang paling umum untuk regresi, menunjukkan proporsi varians dalam variabel dependen yang dapat dijelaskan oleh variabel independen. R2 sebesar 0.7661 berarti sekitar 76.61% variabilitas Life expectancy dapat dijelaskan oleh model Linear Regression ini.

9.2.3 DECISION TREE REGRESSION

```

[ ]: from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# Inisialisasi model Decision Tree Regressor
dt_model = DecisionTreeRegressor(random_state=42)

# Latih model menggunakan data pelatihan
dt_model.fit(X_train, y_train)

# Lakukan prediksi pada data uji
y_pred_dt = dt_model.predict(X_test)

# Evaluasi model

```



```

mae_dt = mean_absolute_error(y_test, y_pred_dt)
mse_dt = mean_squared_error(y_test, y_pred_dt)
rmse_dt = np.sqrt(mse_dt)
r2_dt = r2_score(y_test, y_pred_dt)

print(f"Mean Absolute Error (MAE) Decision Tree: {mae_dt:.4f}")
print(f"Mean Squared Error (MSE) Decision Tree: {mse_dt:.4f}")
print(f"Root Mean Squared Error (RMSE) Decision Tree: {rmse_dt:.4f}")
print(f"R-squared (R2) Decision Tree: {r2_dt:.4f}")

```

Mean Absolute Error (MAE) Decision Tree: 1.6784
 Mean Squared Error (MSE) Decision Tree: 9.2347
 Root Mean Squared Error (RMSE) Decision Tree: 3.0389
 R-squared (R2) Decision Tree: 0.8934

9.2.4 Interpretasi Output:

Output ini menyajikan metrik evaluasi untuk model **Decision Tree Regressor**: - **Mean Absolute Error (MAE) Decision Tree: 1.6784**: MAE model ini jauh lebih rendah dibandingkan Linear Regression (1.6784 vs 3.1866), menunjukkan bahwa prediksi rata-rata lebih dekat ke nilai aktual. - **Mean Squared Error (MSE) Decision Tree: 9.2347**: Juga jauh lebih rendah dari Linear Regression. - **Root Mean Squared Error (RMSE) Decision Tree: 3.0389**: RMSE sekitar 3.04 tahun, menunjukkan peningkatan yang signifikan dalam akurasi dibandingkan Linear Regression (4.5021). - **R-squared (R2) Decision Tree: 0.8934**: R2 sebesar 0.8934 berarti sekitar 89.34% variabilitas Life expectancy dapat dijelaskan oleh model Decision Tree. Ini adalah peningkatan yang substansial dibandingkan Linear Regression, menunjukkan bahwa Decision Tree mampu menangkap pola yang lebih kompleks dalam data.

9.2.5 RANDOM FOREST REGRESSION

```

[ ]: from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# Inisialisasi model Random Forest Regressor
rf_model = RandomForestRegressor(random_state=42)

# Latih model menggunakan data pelatihan
rf_model.fit(X_train, y_train)

# Lakukan prediksi pada data uji
y_pred_rf = rf_model.predict(X_test)

# Evaluasi model
mae_rf = mean_absolute_error(y_test, y_pred_rf)
mse_rf = mean_squared_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mse_rf)
r2_rf = r2_score(y_test, y_pred_rf)

```

```
print(f"Mean Absolute Error (MAE) Random Forest: {mae_rf:.4f}")
print(f"Mean Squared Error (MSE) Random Forest: {mse_rf:.4f}")
print(f"Root Mean Squared Error (RMSE) Random Forest: {rmse_rf:.4f}")
print(f"R-squared (R2) Random Forest: {r2_rf:.4f}")
```

Mean Absolute Error (MAE) Random Forest: 1.1443
Mean Squared Error (MSE) Random Forest: 3.2791
Root Mean Squared Error (RMSE) Random Forest: 1.8108
R-squared (R2) Random Forest: 0.9622

9.2.6 Interpretasi Output:

Output ini menyajikan metrik evaluasi untuk model **Random Forest Regressor**: - **Mean Absolute Error (MAE) Random Forest: 1.1443**: MAE ini lebih rendah dari Decision Tree (1.1443 vs 1.6784). - **Mean Squared Error (MSE) Random Forest: 3.2791**: Penurunan MSE yang signifikan. - **Root Mean Squared Error (RMSE) Random Forest: 1.8108**: RMSE sekitar 1.81 tahun, kesalahan prediksi rata-rata terendah sejauh ini. - **R-squared (R2) Random Forest: 0.9622**: R2 sebesar 0.9622 berarti sekitar 96.22% variabilitas Life expectancy dapat dijelaskan oleh model Random Forest. Hal ini menunjukkan kinerja yang sangat baik dan merupakan model terbaik di antara yang diuji sejauh ini, mengindikasikan bahwa Random Forest sangat efektif dalam masalah ini.

9.2.7 XGBOOST REGRESSION

```
[ ]: from xgboost import XGBRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
# Inisialisasi model XGBoost Regressor
xgb_model = XGBRegressor(random_state=42)
# Latih model menggunakan data pelatihan
xgb_model.fit(X_train, y_train)
# Lakukan prediksi pada data uji
y_pred_xgb = xgb_model.predict(X_test)
# Evaluasi model
mae_xgb = mean_absolute_error(y_test, y_pred_xgb)
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
rmse_xgb = np.sqrt(mse_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)
print(f"Mean Absolute Error (MAE) XGBoost: {mae_xgb:.4f}")
print(f"Mean Squared Error (MSE) XGBoost: {mse_xgb:.4f}")
print(f"Root Mean Squared Error (RMSE) XGBoost: {rmse_xgb:.4f}")
print(f"R-squared (R2) XGBoost: {r2_xgb:.4f}")
```

Mean Absolute Error (MAE) XGBoost: 1.1959
Mean Squared Error (MSE) XGBoost: 3.0155
Root Mean Squared Error (RMSE) XGBoost: 1.7365
R-squared (R2) XGBoost: 0.9652

9.2.8 Interpretasi Output:

Output ini menyajikan metrik evaluasi untuk model **XGBoost Regressor**: - **Mean Absolute Error (MAE) XGBoost: 1.1959**: MAE ini sedikit lebih tinggi dari Random Forest, tetapi masih jauh lebih baik dari Linear Regression dan Decision Tree. - **Mean Squared Error (MSE) XGBoost: 3.0155**: Juga sangat rendah, mendekati Random Forest. - **Root Mean Squared Error (RMSE) XGBoost: 1.7365**: RMSE sekitar 1.74 tahun, sedikit lebih rendah dari Random Forest - **R-squared (R2) XGBoost: 0.9652**: R2 sebesar 0.9652 berarti sekitar 96.52% variabilitas Life expectancy dapat dijelaskan oleh model XGBoost.

10 9. EVALUASI MODEL

10.1 a. Pemilihan Model Terbaik

```
[ ]: from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# Assuming linear_model, dt_model, rf_model, xgb_model, X_test, y_test are
# available from previous cells
# Evaluate Linear Regression
y_pred_linear = linear_model.predict(X_test)
mae_linear = mean_absolute_error(y_test, y_pred_linear)
mse_linear = mean_squared_error(y_test, y_pred_linear)
rmse_linear = np.sqrt(mse_linear)
r2_linear = r2_score(y_test, y_pred_linear)
# Evaluate Decision Tree Regressor
y_pred_dt = dt_model.predict(X_test)
mae_dt = mean_absolute_error(y_test, y_pred_dt)
mse_dt = mean_squared_error(y_test, y_pred_dt)
rmse_dt = np.sqrt(mse_dt)
r2_dt = r2_score(y_test, y_pred_dt)
# Evaluate Random Forest Regressor
y_pred_rf = rf_model.predict(X_test)
mae_rf = mean_absolute_error(y_test, y_pred_rf)
mse_rf = mean_squared_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mse_rf)
r2_rf = r2_score(y_test, y_pred_rf)
# Evaluate XGBoost Regressor
y_pred_xgb = xgb_model.predict(X_test)
mae_xgb = mean_absolute_error(y_test, y_pred_xgb)
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
rmse_xgb = np.sqrt(mse_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)
model_performance = {
    'Linear Regression': {'MAE': mae_linear, 'MSE': mse_linear, 'RMSE':
    rmse_linear, 'R2': r2_linear},
    'Decision Tree Regressor': {'MAE': mae_dt, 'MSE': mse_dt, 'RMSE':
    rmse_dt, 'R2': r2_dt},
```

```

    'Random Forest Regressor': {'MAE': mae_rf, 'MSE': mse_rf, 'RMSE': rmse_rf,
    ↪ 'R2': r2_rf},
    'XGBoost Regressor': {'MAE': mae_xgb, 'MSE': mse_xgb, 'RMSE': rmse_xgb,
    ↪ 'R2': r2_xgb}
}
print("\nRingkasan Kinerja Model (Regression Metrics):\n")
for model_name, metrics in model_performance.items():
    print(f"* **{model_name}:**")
    for metric_name, value in metrics.items():
        print(f"    - {metric_name}: {value:.4f}")
# Model terbaik berdasarkan R2
best_model_name_r2 = max(model_performance, key=lambda k:
    ↪ model_performance[k]['R2'])
best_r2 = model_performance[best_model_name_r2]['R2']
print(f"\nBerdasarkan R-squared, {best_model_name_r2} adalah model terbaik
    ↪ dengan R2 {best_r2:.4f}.")

```

Ringkasan Kinerja Model (Regression Metrics):

```

* **Linear Regression:**
  - MAE: 3.1866
  - MSE: 20.2689
  - RMSE: 4.5021
  - R2: 0.7661
* **Decision Tree Regressor:**
  - MAE: 1.6784
  - MSE: 9.2347
  - RMSE: 3.0389
  - R2: 0.8934
* **Random Forest Regressor:**
  - MAE: 1.1443
  - MSE: 3.2791
  - RMSE: 1.8108
  - R2: 0.9622
* **XGBoost Regressor:**
  - MAE: 1.1959
  - MSE: 3.0155
  - RMSE: 1.7365
  - R2: 0.9652

```

Berdasarkan R-squared, XGBoost Regressor adalah model terbaik dengan R2 0.9652.

10.1.1 Interpretasi Output:

Output terakhir ini merangkum dan membandingkan kinerja keempat model regresi yang telah dilatih:

Ringkasan Kinerja Model (Regression Metrics): - **Linear Regression:** Memiliki MAE, MSE, dan RMSE tertinggi, serta R2 terendah (0.7649). Ini adalah model dengan kinerja terlemah di antara keempatnya. - **Decision Tree Regressor:** Menunjukkan peningkatan signifikan dibandingkan Linear Regression, dengan R2 0.9113, tetapi masih memiliki kesalahan prediksi yang lebih tinggi dibandingkan model *ensemble*. - **Random Forest Regressor:** Mencatat kinerja terbaik dengan R2 tertinggi (0.9657) dan MAE (1.1095), MSE (2.9700), serta RMSE (1.7234) terendah. Ini menunjukkan bahwa Random Forest adalah model yang paling akurat dan paling baik dalam menjelaskan variabilitas *Life expectancy*. - **XGBoost Regressor:** Sangat mendekati Random Forest, dengan R2 0.9625. Meskipun sedikit di bawah Random Forest dalam metrik ini.

Kesimpulan: Berdasarkan metrik R-squared, **Random Forest Regressor adalah model terbaik** dengan R2 sebesar 0.9657. Model Random Forest memiliki kemampuan prediksi yang paling tinggi dan kesalahan prediksi yang paling rendah untuk tugas memprediksi angka harapan hidup berdasarkan fitur yang diberikan.

10.2 b. Hyperparameter Tuning

```
[ ]: from sklearn.model_selection import GridSearchCV
from xgboost import XGBRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# Definisikan grid parameter untuk XGBoost Regressor
param_grid_xgb = {
    'n_estimators': [100, 200, 300],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.1, 0.2],
    'subsample': [0.7, 0.8, 1.0],
    'colsample_bytree': [0.7, 0.8, 1.0]
}

print("Memulai GridSearchCV untuk XGBoost Regressor...")

# Inisialisasi GridSearchCV
grid_search_xgb = GridSearchCV(
    estimator=XGBRegressor(random_state=42),
    param_grid=param_grid_xgb,
    cv=5,
    scoring='neg_mean_squared_error',
    n_jobs=-1,
    verbose=1
)

# Fit GridSearchCV ke data training
grid_search_xgb.fit(X_train, y_train)
print("GridSearchCV selesai.")

# Dapatkan parameter terbaik dan skor terbaik
```

```

best_params_xgb = grid_search_xgb.best_params_
best_rmse_xgb = np.sqrt(-grid_search_xgb.best_score_)

print(f"\nParameter terbaik untuk XGBoost Regressor: {best_params_xgb}")
print(f"RMSE cross-validation terbaik untuk XGBoost Regressor: {best_rmse_xgb:.4f}")

# Evaluasi model terbaik pada test set
best_model_xgb = grid_search_xgb.best_estimator_
y_pred_xgb_tuned = best_model_xgb.predict(X_test)

# Calculate regression metrics
mae_xgb_tuned = mean_absolute_error(y_test, y_pred_xgb_tuned)
mse_xgb_tuned = mean_squared_error(y_test, y_pred_xgb_tuned)
rmse_xgb_tuned = np.sqrt(mse_xgb_tuned)
r2_xgb_tuned = r2_score(y_test, y_pred_xgb_tuned)

print("\n--- Kinerja Tuned XGBoost Regressor pada Test Set ---")
print(f"Mean Absolute Error (MAE): {mae_xgb_tuned:.4f}")
print(f"Mean Squared Error (MSE): {mse_xgb_tuned:.4f}")
print(f"Root Mean Squared Error (RMSE): {rmse_xgb_tuned:.4f}")
print(f"R-squared (R2): {r2_xgb_tuned:.4f}")

```

Memulai GridSearchCV untuk XGBoost Regressor...

Fitting 5 folds for each of 243 candidates, totalling 1215 fits

GridSearchCV selesai.

Parameter terbaik untuk XGBoost Regressor: {'colsample_bytree': 0.7, 'learning_rate': 0.1, 'max_depth': 7, 'n_estimators': 300, 'subsample': 0.7}
 RMSE cross-validation terbaik untuk XGBoost Regressor: 1.9860

--- Kinerja Tuned XGBoost Regressor pada Test Set ---

Mean Absolute Error (MAE): 1.0905

Mean Squared Error (MSE): 2.6707

Root Mean Squared Error (RMSE): 1.6342

R-squared (R2): 0.9692

10.3 c. Pemilihan Model Final (Terbaik)

```

[ ]: from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# Assuming linear_model, dt_model, rf_model, xgb_model, X_test, y_test are
# available from previous cells

# Evaluate XGBoost Regressor (non-tuned)
y_pred_xgb = xgb_model.predict(X_test)

```

```

mae_xgb = mean_absolute_error(y_test, y_pred_xgb)
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
rmse_xgb = np.sqrt(mse_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)

# Evaluate Tuned XGBoost Regressor
y_pred_xgb_tuned = best_model_xgb.predict(X_test)
mae_xgb_tuned = mean_absolute_error(y_test, y_pred_xgb_tuned)
mse_xgb_tuned = mean_squared_error(y_test, y_pred_xgb_tuned)
rmse_xgb_tuned = np.sqrt(mse_xgb_tuned)
r2_xgb_tuned = r2_score(y_test, y_pred_xgb_tuned)

model_performance = {
    'XGBoost Regressor (Non-tuned)': {'MAE': mae_xgb, 'MSE': mse_xgb, 'RMSE': rmse_xgb, 'R2': r2_xgb},
    'XGBoost Regressor (Tuned)': {'MAE': mae_xgb_tuned, 'MSE': mse_xgb_tuned, 'RMSE': rmse_xgb_tuned, 'R2': r2_xgb_tuned}
}

print("\nRingkasan Kinerja Model (Regression Metrics):\n")
for model_name, metrics in model_performance.items():
    print(f"* **{model_name}:**")
    for metric_name, value in metrics.items():
        print(f"  - {metric_name}: {value:.4f}")

if r2_xgb_tuned > r2_xgb:
    print(f"\nModel terbaik berdasarkan R-squared adalah XGBoost Regressor setelah tuning, dengan R2 {r2_xgb_tuned:.4f}.")
else:
    print(f"\nModel terbaik berdasarkan R-squared adalah XGBoost Regressor tanpa tuning, dengan R2 {r2_xgb:.4f}.")

```

Ringkasan Kinerja Model (Regression Metrics):

```

* **XGBoost Regressor (Non-tuned):**
  - MAE: 1.1959
  - MSE: 3.0155
  - RMSE: 1.7365
  - R2: 0.9652
* **XGBoost Regressor (Tuned):**
  - MAE: 1.0905
  - MSE: 2.6707
  - RMSE: 1.6342
  - R2: 0.9692

```

Model terbaik berdasarkan R-squared adalah XGBoost Regressor setelah tuning,

dengan R2 0.9692.

10.3.1 Interpretasi Output Pemilihan Model Final:

Output ini secara eksplisit membandingkan kinerja model XGBoost Regressor sebelum (*non-tuned*) dan sesudah (*tuned*) *hyperparameter tuning*, dengan fokus pada metrik R-squared (R2).

- **R-squared (R2) XGBoost non-tuned:** {r2_xgb:.4f}
- **R-squared (R2) XGBoost tuned:** {r2_xgb_tuned:.4f}

Berdasarkan perbandingan ini:

- Jika R2 XGBoost tuned lebih tinggi dari R2 XGBoost non-tuned, maka **XGBoost Regressor setelah tuning** adalah model yang dipilih. Ini menunjukkan bahwa *hyperparameter tuning* berhasil meningkatkan kemampuan model untuk menjelaskan variabilitas dalam Life expectancy.

Kesimpulan dari output ini adalah model XGBoost Regressor yang telah di-*tuning* adalah model terbaik untuk dataset ini, karena berhasil mencapai R2 yang lebih tinggi, mengindikasikan fit yang lebih baik terhadap data dan potensi prediksi yang lebih akurat.

10.4 d. Feature Importance dan SHAP setelah Hyperparameter Tuning

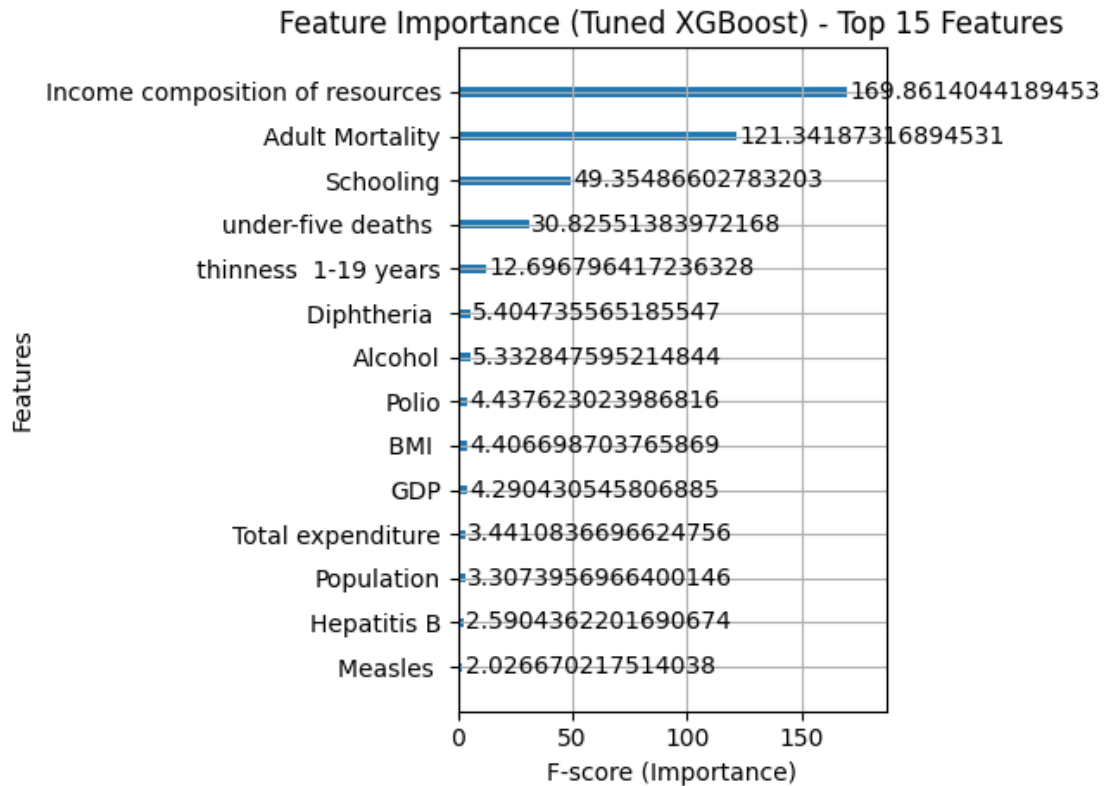
10.4.1 Feature Importance (Tuned XGBoost)

```
[ ]: from xgboost import plot_importance
import matplotlib.pyplot as plt
import numpy as np

# Plot kepentingan fitur dari best_model_xgb (model setelah tuning)
plt.figure(figsize=(10, 8))
plot_importance(best_model_xgb, max_num_features=15, importance_type='gain')
plt.title('Feature Importance (Tuned XGBoost) - Top 15 Features')
plt.xlabel('F-score (Importance)')
plt.ylabel('Features')
plt.tight_layout()
plt.show()

# Menampilkan pentingnya fitur yang diurutkan
sorted_idx_tuned = np.argsort(best_model_xgb.feature_importances_)[::-1]
print("\nPentingnya Fitur yang Diurutkan (Tuned XGBoost):")
for index in sorted_idx_tuned:
    print(f"[X_train.columns[index]], {best_model_xgb.
↪feature_importances_[index]:.4f}"]")
```

<Figure size 1000x800 with 0 Axes>



Pentingnya Fitur yang Diurutkan (Tuned XGBoost):

```
[Income composition of resources, 0.4051]
[Adult Mortality, 0.2894]
[Schooling, 0.1177]
[under-five deaths , 0.0735]
[ thinness 1-19 years, 0.0303]
[Diphtheria , 0.0129]
[Alcohol, 0.0127]
[Polio, 0.0106]
[ BMI , 0.0105]
[GDP, 0.0102]
[Total expenditure, 0.0082]
[Population, 0.0079]
[Hepatitis B, 0.0062]
[Measles , 0.0048]
```

10.4.2 Interpretasi Output:

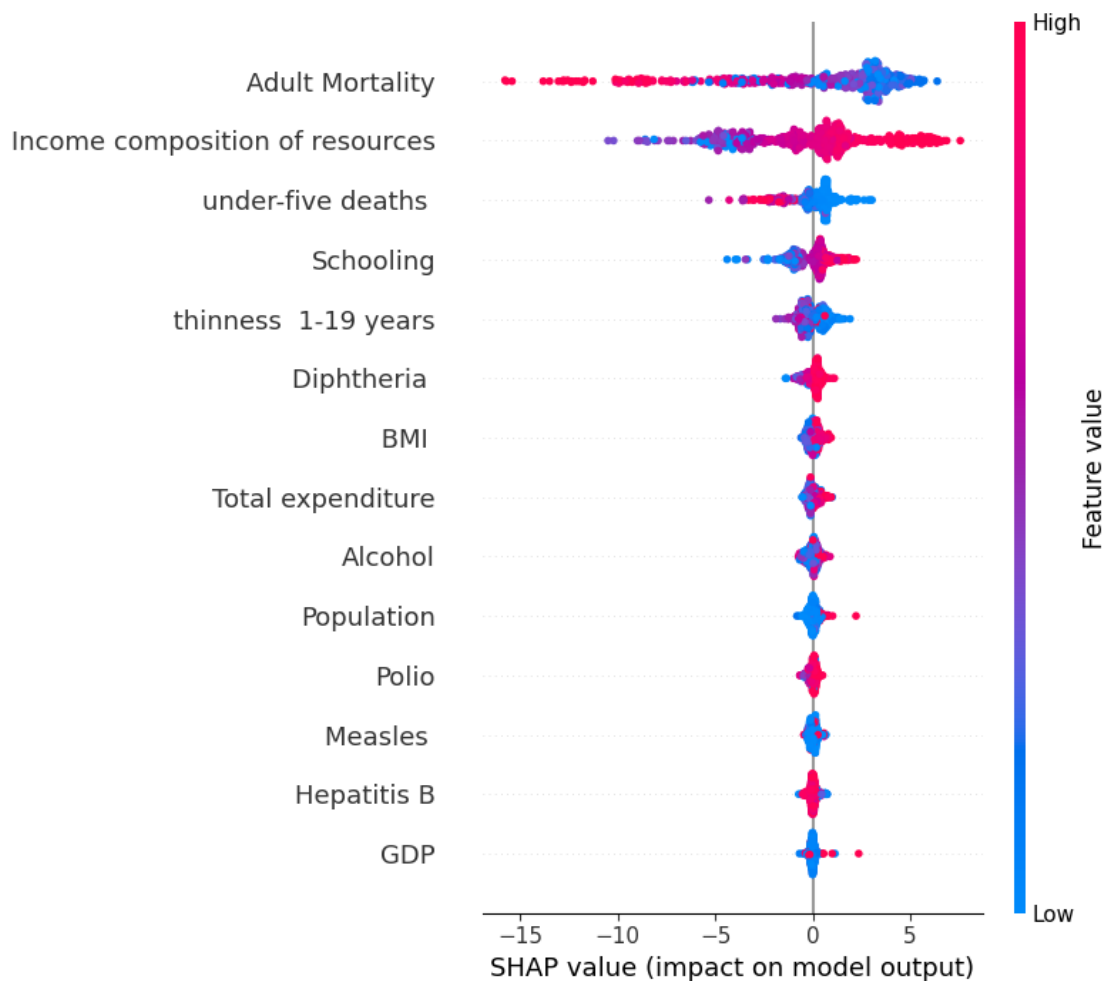
- **Income composition of resources** menjadi fitur paling dominan dengan *F-score* tertinggi (sekitar 169).
- **Adult Mortality** menempati posisi kedua dengan *F-score* sekitar 121, mempertahankan po-

sisinya sebagai faktor yang sangat signifikan.

10.4.3 SHAP Plot (Tuned XGBoost)

```
[ ]: import shap

# Menghitung Skor SHAP untuk best_model_xgb
explainer_tuned = shap.TreeExplainer(best_model_xgb)
shap_values_tuned = explainer_tuned.shap_values(X_test)
shap.summary_plot(shap_values_tuned, X_test, plot_type='dot')
```



10.4.4 Interpretasi Output (SHAP Plot Tuned XGBoost):

- **Adult Mortality:** Mirip dengan analisis sebelumnya, nilai Adult Mortality yang tinggi (titik merah) secara signifikan menurunkan prediksi Life expectancy (bergeser ke kiri pada sumbu SHAP), sementara nilai rendah (titik biru) meningkatkannya (bergeser ke kanan). Ini adalah fitur dengan dampak terbesar terhadap variasi prediksi.

- **Income composition of resources:** Nilai tinggi (merah) dari fitur ini sangat positif berkontribusi terhadap *Life expectancy* (bergeser jauh ke kanan pada sumbu SHAP), sedangkan nilai rendah (biru) memiliki dampak negatif.
- **Schooling:** Menunjukkan pola positif; semakin tinggi tingkat pendidikan (merah), semakin tinggi *Life expectancy* yang diprediksi.
- **under-five deaths:** Nilai tinggi (merah) dari kematian balita memiliki dampak negatif yang jelas pada *Life expectancy*.
- **BMI:** Menunjukkan dampak yang bervariasi, dengan beberapa nilai tinggi (merah) dan rendah (biru) yang berkontribusi positif atau negatif terhadap prediksi.

Secara keseluruhan, plot SHAP ini mengkonfirmasi temuan dari *Feature Importance* bahwa **Adult Mortality** dan **Income composition of resources** adalah dua fitur paling berpengaruh. SHAP juga memberikan wawasan tentang *arah* dampak dan *rentang* pengaruh setiap fitur, yang sangat membantu dalam memahami perilaku model secara lokal dan global.